

Name:- Tanmay Santosh Sanap

Roll No:-40

Student Id:- 5144978

College:- B K BIRLA COLLEGE

Subject:-Data Engineering (Practical No :- 01)

1) Parsing and extracting data from text, CSV, HTML, XML, and JSON files.

```
In [ ]: import csv
import json
import xml.etree.ElementTree as ET
from bs4 import BeautifulSoup
import pandas as pd

# --- 1.1 Text and CSV Files ---
# Create a sample CSV file
csv_data = '''id,name,age,city
1,Alice,30,New York
2,Bob,25,Los Angeles
3,Charlie,35,Chicago'''

with open('sample.csv', 'w') as f:
    f.write(csv_data)

# Read CSV using pandas
df = pd.read_csv('sample.csv')
print("--- CSV Data with Pandas ---")
display(df)

# --- 1.2 JSON Files ---
# Create a sample JSON file
json_data = {
    "company": "Tech Corp",
    "employees": [
        {"name": "Alice", "role": "Developer"},
        {"name": "Bob", "role": "Designer"}
    ]
}

with open('sample.json', 'w') as f:
    json.dump(json_data, f, indent=4)

# Load and parse JSON
with open('sample.json', 'r') as f:
    parsed_json = json.load(f)
print("\n--- JSON Data ---")
print(f"Company: {parsed_json['company']}")
for emp in parsed_json['employees']:
```

```

    print(f" - {emp['name']}: {emp['role']}")

# --- 1.3 XML Files ---
# Create a sample XML file
xml_data = '''<library>
    <book id="1">
        <title>Python 101</title>
        <author>John Doe</author>
    </book>
    <book id="2">
        <title>Data Science Handbook</title>
        <author>Jane Smith</author>
    </book>
</library>'''
with open('sample.xml', 'w') as f:
    f.write(xml_data)

# Parse XML
tree = ET.parse('sample.xml')
root = tree.getroot()
print("\n--- XML Data ---")
for book in root.findall('book'):
    book_id = book.get('id')
    title = book.find('title').text
    author = book.find('author').text
    print(f"Book #{book_id}: '{title}' by {author}")

# --- 1.4 HTML Parsing ---
html_data = '''<html>
    <body>
        <h1>Welcome to Web Scraping</h1>
        <p class="description">Learn to extract data from HTML.</p>
        <ul id="languages">
            <li>Python</li>
            <li>SQL</li>
        </ul>
    </body>
</html>'''

soup = BeautifulSoup(html_data, 'html.parser')
print("\n--- HTML Extract ---")
print("Header:", soup.find('h1').text)
print("Description:", soup.find('p', class_='description').text)
print("Languages:", [li.text for li in soup.find('ul', id='languages').fi

```

--- CSV Data with Pandas ---

	id	name	age	city
0	1	Alice	30	New York
1	2	Bob	25	Los Angeles
2	3	Charlie	35	Chicago

```
--- JSON Data ---
Company: Tech Corp
- Alice: Developer
- Bob: Designer

--- XML Data ---
Book #1: 'Python 101' by John Doe
Book #2: 'Data Science Handbook' by Jane Smith

--- HTML Extract ---
Header: Welcome to Web Scraping
Description: Learn to extract data from HTML.
Languages: ['Python', 'SQL']
```

1. Performing read and write operations on binary files

```
In [ ]: import pickle

# Python object to serialize
data_object = {'user_id': 101, 'scores': [95, 88, 100], 'active': True}

# Write binary (serialization)
with open('data.bin', 'wb') as bin_file:
    pickle.dump(data_object, bin_file)

# Read binary (deserialization)
with open('data.bin', 'rb') as bin_file:
    loaded_object = pickle.load(bin_file)

print("Loaded Binary Object:", loaded_object)

Loaded Binary Object: {'user_id': 101, 'scores': [95, 88, 100], 'active': True}
```

1. Searching, splitting and replacing strings using regular expressions

```
In [ ]: import re

text = "Contact support at support@example.com or sales@example.org. Phon

# 3.1 Search/Find All (Extract email addresses)
emails = re.findall(r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}', text)
print("Extracted Emails:", emails)

# 3.2 Splitting strings (Split by whitespace or punctuation)
split_words = re.split(r'[, \s\.\!]+', text)
print("Split words:", split_words[:6]) # Display first few words

# 3.3 Replacing substrings (Anonymize email domains)
anonymized_text = re.sub(r'@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}', '@hidden.com', text)
print("Anonymized Text:", anonymized_text)
```

Extracted Emails: ['support@example.com', 'sales@example.org']
Split words: ['Contact', 'support', 'at', 'support@example', 'com', 'or']
Anonymized Text: Contact support at support@hidden.com or sales@hidden.com. Phone: 123-456-7890.

1. Creating and managing a relational database using SQL CRUD operations

In []: **import** sqlite3

```
# Connect to an in-memory SQLite database
conn = sqlite3.connect(':memory:')
cursor = conn.cursor()

# --- Create (Table and Insert records) ---
cursor.execute('''
CREATE TABLE products (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    price REAL,
    stock INTEGER
)
''')

cursor.executemany('''
INSERT INTO products (name, price, stock) VALUES (?, ?, ?)
''', [
    ('Laptop', 999.99, 10),
    ('Smartphone', 499.99, 25),
    ('Headphones', 79.99, 50)
])
conn.commit()

# --- Read ---
print("--- Initial Product Catalog ---")
cursor.execute("SELECT * FROM products")
for row in cursor.fetchall():
    print(row)

# --- Update ---
cursor.execute("UPDATE products SET price = ? WHERE name = ?", (449.99, 'Smartphone'))
conn.commit()

# --- Delete ---
cursor.execute("DELETE FROM products WHERE name = ?", ('Headphones',))
conn.commit()

# --- Read modified catalog ---
print("\n--- Updated Product Catalog ---")
cursor.execute("SELECT * FROM products")
for row in cursor.fetchall():
    print(row)

conn.close()
```

```
--- Initial Product Catalog ---  
(1, 'Laptop', 999.99, 10)  
(2, 'Smartphone', 499.99, 25)  
(3, 'Headphones', 79.99, 50)
```

```
--- Updated Product Catalog ---  
(1, 'Laptop', 999.99, 10)  
(2, 'Smartphone', 449.99, 25)
```

Name:- Tanmay Santosh Sanap

Roll No:- 40

Student Id:- 5144978

College:- B K BIRLA COLLEGE

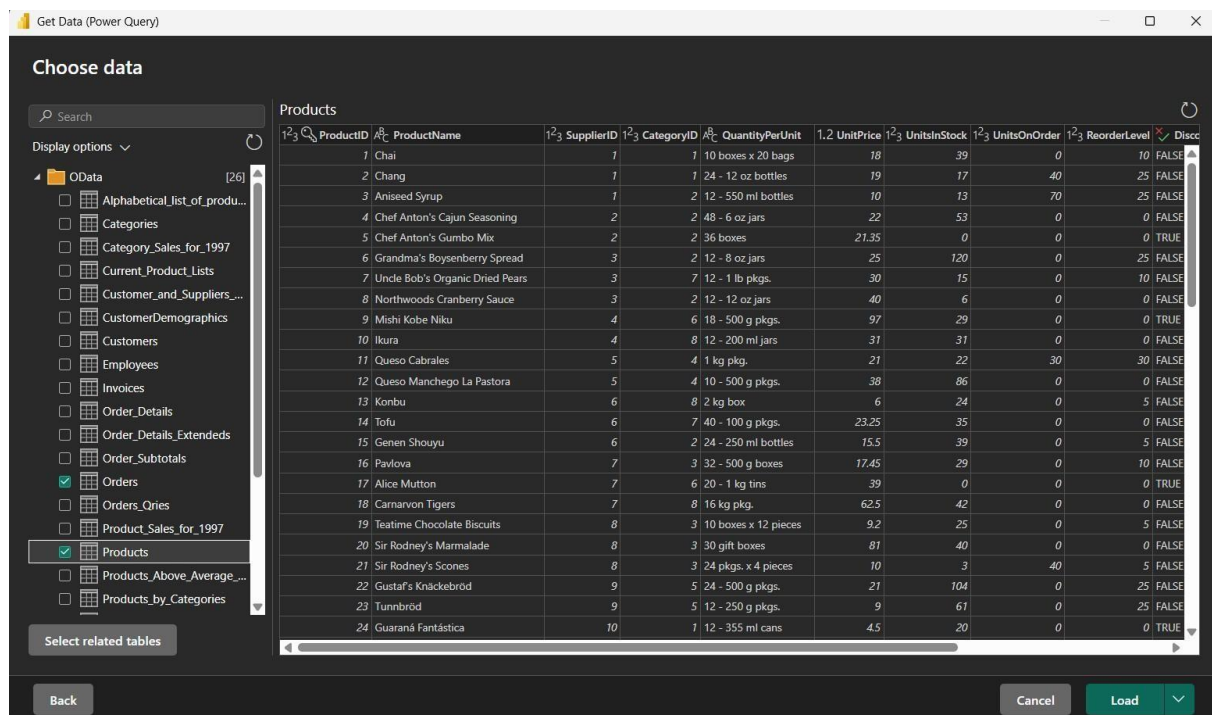
Subject:-Data Engineering (Practical No :- 02)

Aim:

To perform the ETL process in Power BI by extracting, transforming, and loading data to prepare it for analysis.

Step 1- Import Excel Data into Power BI

The Excel workbook containing the Products, Orders, and Order_Details tables was imported into Power BI for further data transformation and analysis.



Step 2- Open Power Query Editor

Power Query Editor was opened to perform the required data transformation operations on the imported tables.

Power Query Editor

Home Transform Add column View Help

Close & Apply, Get data, Enter data, Manage parameters, Refresh, Advanced editor, Export query results, Choose columns, Remove columns, Keep rows, Remove rows, Filter rows, Sort, Transform, Combine, Export template

Queries [2]: Orders, Products

Table.ReorderColumns(#"Added custom", {"OrderID", "CustomerID", "EmployeeID", "OrderDate", "RequiredDate", "ShippedDate", "ShipVia"})

	OrderID	CustomerID	EmployeeID	OrderDate	RequiredDate	ShippedDate	ShipVia
1	10248	VINET	5	7/4/1996, 12:00:00 AM	8/1/1996, 12:00:00 AM	7/16/1996, 12:00:00 AM	3
2	10248	VINET	5	7/4/1996, 12:00:00 AM	8/1/1996, 12:00:00 AM	7/16/1996, 12:00:00 AM	3
3	10248	VINET	5	7/4/1996, 12:00:00 AM	8/1/1996, 12:00:00 AM	7/16/1996, 12:00:00 AM	3
4	10249	TOMSP	6	7/5/1996, 12:00:00 AM	8/16/1996, 12:00:00 AM	7/10/1996, 12:00:00 AM	1
5	10249	TOMSP	6	7/5/1996, 12:00:00 AM	8/16/1996, 12:00:00 AM	7/10/1996, 12:00:00 AM	1
6	10250	HANAR	4	7/8/1996, 12:00:00 AM	8/5/1996, 12:00:00 AM	7/12/1996, 12:00:00 AM	2
7	10250	HANAR	4	7/8/1996, 12:00:00 AM	8/5/1996, 12:00:00 AM	7/12/1996, 12:00:00 AM	2
8	10250	HANAR	4	7/8/1996, 12:00:00 AM	8/5/1996, 12:00:00 AM	7/12/1996, 12:00:00 AM	2
9	10251	VICTE	3	7/8/1996, 12:00:00 AM	8/5/1996, 12:00:00 AM	7/15/1996, 12:00:00 AM	1
10	10251	VICTE	3	7/8/1996, 12:00:00 AM	8/5/1996, 12:00:00 AM	7/15/1996, 12:00:00 AM	1
11	10251	VICTE	3	7/8/1996, 12:00:00 AM	8/5/1996, 12:00:00 AM	7/15/1996, 12:00:00 AM	1
12	10252	SUPRD	4	7/9/1996, 12:00:00 AM	8/6/1996, 12:00:00 AM	7/11/1996, 12:00:00 AM	2
13	10252	SUPRD	4	7/9/1996, 12:00:00 AM	8/6/1996, 12:00:00 AM	7/11/1996, 12:00:00 AM	2
14	10252	SUPRD	4	7/9/1996, 12:00:00 AM	8/6/1996, 12:00:00 AM	7/11/1996, 12:00:00 AM	2
15	10253	HANAR	3	7/10/1996, 12:00:00 AM	7/24/1996, 12:00:00 AM	7/16/1996, 12:00:00 AM	2
16	10253	HANAR	3	7/10/1996, 12:00:00 AM	7/24/1996, 12:00:00 AM	7/16/1996, 12:00:00 AM	2
17	10253	HANAR	3	7/10/1996, 12:00:00 AM	7/24/1996, 12:00:00 AM	7/16/1996, 12:00:00 AM	2
18	10254	CHOPS	5	7/11/1996, 12:00:00 AM	8/8/1996, 12:00:00 AM	7/23/1996, 12:00:00 AM	2
19	10254	CHOPS	5	7/11/1996, 12:00:00 AM	8/8/1996, 12:00:00 AM	7/23/1996, 12:00:00 AM	2
20	10254	CHOPS	5	7/11/1996, 12:00:00 AM	8/8/1996, 12:00:00 AM	7/23/1996, 12:00:00 AM	2
21	10255	RICSU	9	7/12/1996, 12:00:00 AM	8/9/1996, 12:00:00 AM	7/15/1996, 12:00:00 AM	3
22	10255	RICSU	9	7/12/1996, 12:00:00 AM	8/9/1996, 12:00:00 AM	7/15/1996, 12:00:00 AM	3

Completed (0.81 s) Columns: 21 Rows: 99+

Query settings: Properties, Name: Orders, Applied steps: Source, Navigation 1, Expanded..., Added cust..., Reordered..., Data destination: No data destination

Step 3- Remove Unnecessary Columns

Unnecessary columns were removed from the Products table to retain only the fields required for further analysis.

Power Query Editor

Home Transform Add column View Help

Close & Apply, Get data, Enter data, Manage parameters, Refresh, Advanced editor, Export query results, Choose columns, Remove columns, Keep rows, Remove rows, Filter rows, Sort, Transform, Combine, Export template

Queries [2]: Orders, Products

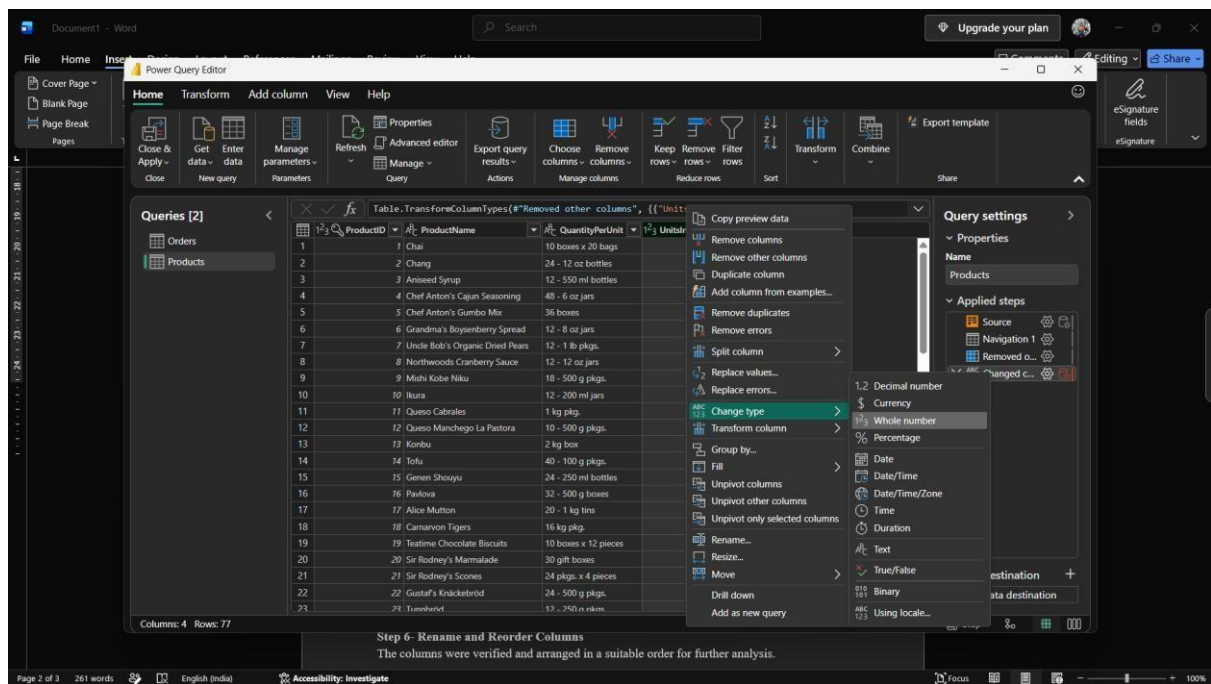
Table.TransformColumnTypes(#"Removed other columns", ({"UnitsInStock", Int64.Type}))

	ProductID	ProductName	QuantityPerUnit	UnitsInStock
1	1	Chai	10 boxes x 20 bags	39
2	2	Chang	24 - 12 oz bottles	17
3	3	Aniseed Syrup	12 - 550 ml bottles	13
4	4	Chef Anton's Cajun Seasoning	48 - 6 oz jars	53
5	5	Chef Anton's Gumbo Mix	36 boxes	0
6	6	Grandma's Boysenberry Spread	12 - 8 oz jars	120
7	7	Uncle Bob's Organic Dried Pears	12 - 1 lb pkgs.	15
8	8	Northwoods Cranberry Sauce	12 - 12 oz jars	6
9	9	Mishi Kobe Niku	18 - 500 g pkgs.	29
10	10	Ikura	12 - 200 ml jars	31
11	11	Queso Cabrales	1 kg pkg.	22
12	12	Queso Manchego La Pastora	10 - 500 g pkgs.	86
13	13	Konbu	2 kg box	24
14	14	Tofu	40 - 100 g pkgs.	35
15	15	Genen Shouyu	24 - 250 ml bottles	39
16	16	Pavlova	32 - 500 g boxes	29
17	17	Alice Mutton	20 - 1 kg tins	0
18	18	Carnarvon Tigers	16 kg pkg.	42
19	19	Teatime Chocolate Biscuits	10 boxes x 12 pieces	25
20	20	Sir Rodney's Marmalade	30 gift boxes	40
21	21	Sir Rodney's Scones	24 pkgs. x 4 pieces	3
22	22	Gustaf's Knäckebröd	24 - 500 g pkgs.	104
23	23	Tunnbröd	12 - 250 g pkgs.	61

Query settings: Properties, Name: Products, Applied steps: Source, Navigation 1, Removed o..., Changed c..., Data destination: No data destination

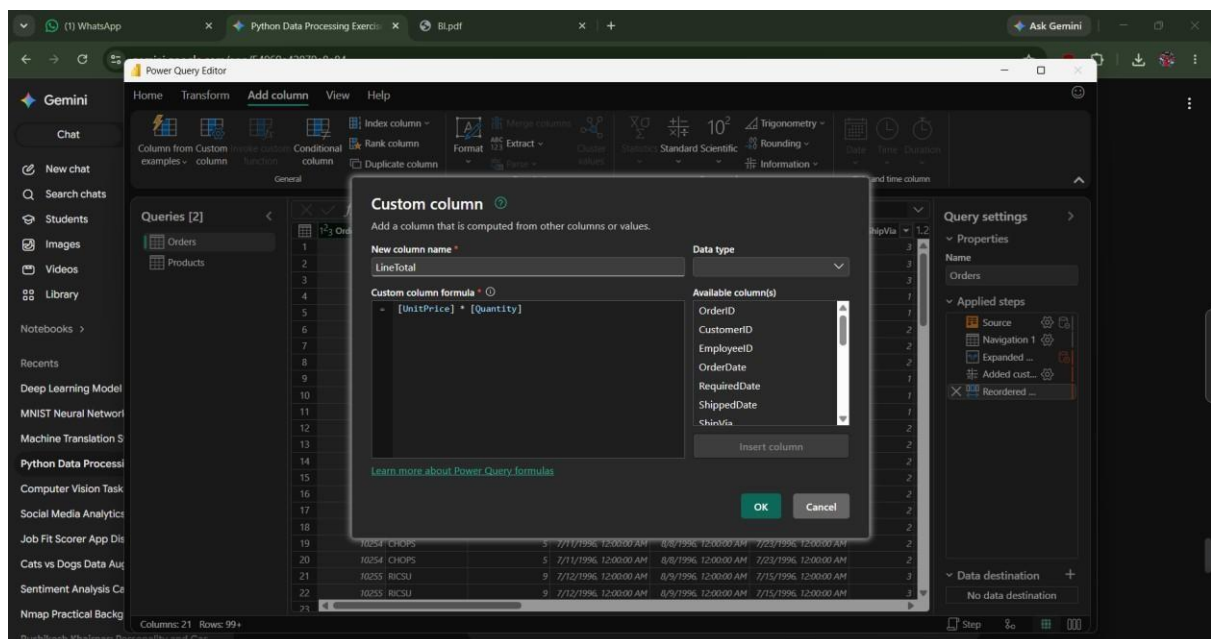
Step 4 - Verify the Data Type of UnitsInStock

The data type of the UnitsInStock column was verified as Whole Number, which is appropriate for representing stock quantities.



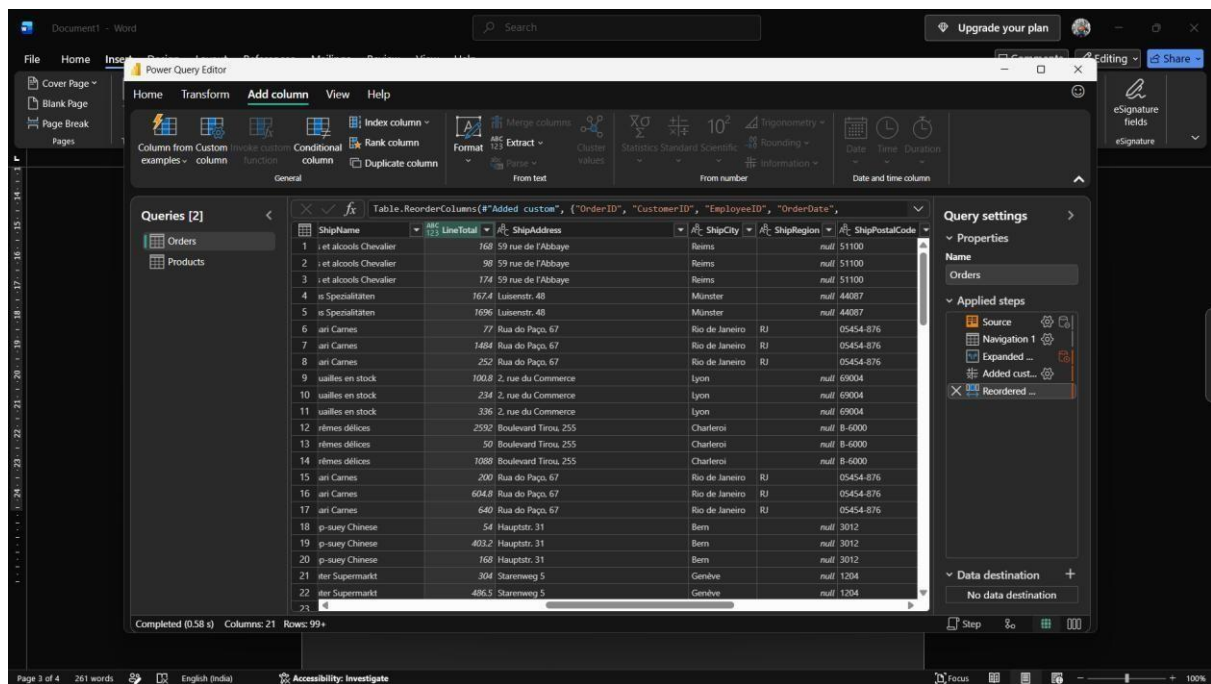
Step 5- Calculate Line Total

A new LineTotal column was created by multiplying UnitPrice by Quantity for each order detail.



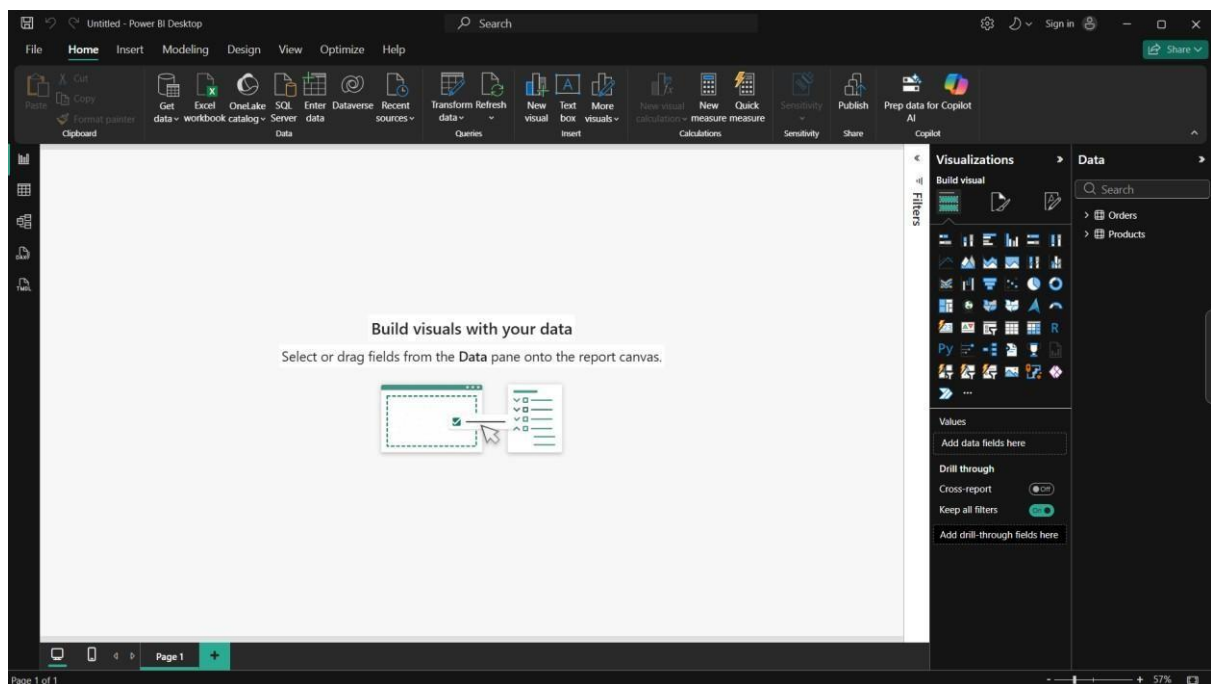
Step 6- Rename and Reorder Columns

The columns were verified and arranged in a suitable order for further analysis.



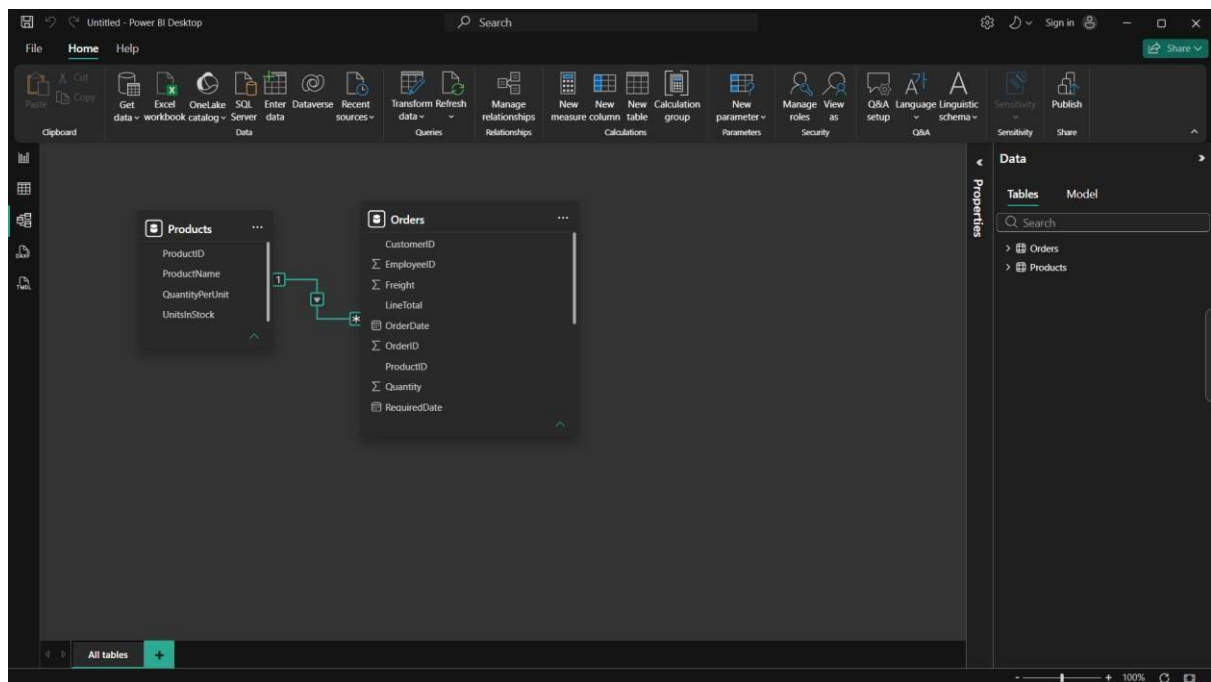
Step 7- Load the Transformed Data

The transformed data was loaded into Power BI Desktop using the Close & Apply option.



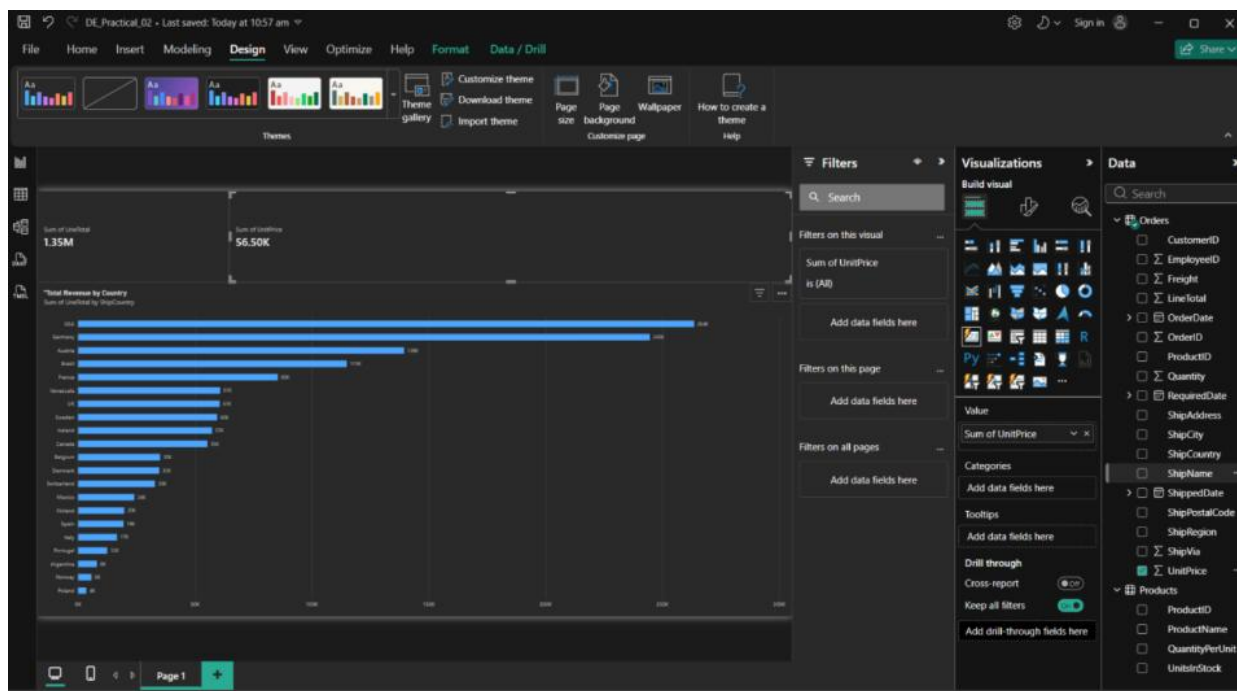
Step 8 Create Relationships Between Tables

Relationships were created between the related tables using their common key fields to establish the data model.



Step 9 Result

The ETL process was successfully performed in Power BI by extracting data from an Excel workbook, transforming the data using Power Query, creating the Line Total column, loading the transformed data, and establishing relationships between the tables.



Name:- Tanmay Santosh Sanap

Roll No:- 40

Student Id:- 5144978

College:- B K BIRLA COLLEGE

Subject:- Data Engineering (Practical No :- 03)

Aim:

To perform single and multiple document insertion and retrieval using MongoDB shell (mongosh).

Step 1: Insert Single Document Query:

db.items.insertOne({ name: "laptop", price: 999 });

Output:

```
>_MONGOSH
> db.items.insertOne({ name: "laptop", price: 999 });
< {
  acknowledged: true,
  insertedId: ObjectId('6ac649038fa08d8f4561c384')
}
test>
```

Step 2: View Inserted Document

Query:

db.items.find();

Output:

```
> db.items.find();
< {
  _id: ObjectId('6ac51e1cd4bbf249c034346a'),
  name: 'laptop',
  price: 999
}
{
  _id: ObjectId('6ac649038fa08d8f4561c384'),
  name: 'laptop',
  price: 999
}
test> |
```

Step 3: Insert Multiple Documents

Query:

```
db.products.insertMany([
  { name: "phone", price: 500, stock: 10 },
  { name: "tablet", price: 300, stock: 5 },
  { name: "watch", price: 150, stock: 0 }
]);
```

Output:

```
> db.products.insertMany([
  { name: "phone", price: 500, stock: 10 },
  { name: "tablet", price: 300, stock: 5 },
  { name: "watch", price: 150, stock: 0 }
]);
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ac6498b8fa08d8f4561c385'),
    '1': ObjectId('6ac6498b8fa08d8f4561c386'),
    '2': ObjectId('6ac6498b8fa08d8f4561c387')
  }
}
test>
```

Step 4: View Products in Formatted JSON

Query:

```
db.products.find().pretty();
```

Output:

```
> db.products.find().pretty();
< {
  _id: ObjectId('6ac51e43d4bbf249c034346b'),
  name: 'phone',
  price: 500,
  stock: 10
}
{
  _id: ObjectId('6ac51e43d4bbf249c034346c'),
  name: 'tablet',
  price: 300,
  stock: 5
}
{
  _id: ObjectId('6ac51e43d4bbf249c034346d'),
  name: 'watch',
  price: 150,
  stock: 0
}
{
  _id: ObjectId('6ac6498b8fa08d8f4561c385'),
  name: 'phone',
  price: 500,
  stock: 10
}
```

```
{
  _id: ObjectId('6ac6498b8fa08d8f4561c386'),
  name: 'tablet',
  price: 300,
  stock: 5
}
{
  _id: ObjectId('6ac6498b8fa08d8f4561c387'),
  name: 'watch',
  price: 150,
  stock: 0
}
test>
```

Name:- Tanmay Santosh Sanap

Roll No:-40

Student Id:- 5144978

College:- B K BIRLA COLLEGE

Subject:-Data Engineering (Practical No :- 04)

To perform noise elimination, feature selection, and exploratory data analysis (EDA) using Python libraries.

```
In [4]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.feature_selection import VarianceThreshold

# -----
# Sample Dataset
# -----

data = {
    'Age': [20, 21, 22, 23, 24, 100],          # 100 is an outlier
    'Marks': [75, 80, 85, 90, 95, 20],
    'Attendance': [85, 88, 90, 92, 94, 50],
    'Constant': [1, 1, 1, 1, 1, 1]           # Constant feature
}

df = pd.DataFrame(data)

print("Original Dataset:")
print(df)

# =====
# 1. Noise Elimination (Outlier Removal)
# =====

Q1 = df['Age'].quantile(0.25)
Q3 = df['Age'].quantile(0.75)

IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

print("\nQ1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower Limit:", lower)
```

```

print("Upper Limit:", upper)

# Remove Age outlier
df_clean = df[
    (df['Age'] >= lower) &
    (df['Age'] <= upper)
]

print("\nDataset after Noise Elimination:")
print(df_clean)

# =====
# 2. Feature Selection
# Remove Constant / Low Variance Features
# =====

selector = VarianceThreshold(threshold=0.0)

selected = selector.fit_transform(df_clean)

selected_columns = df_clean.columns[
    selector.get_support()
]

print("\nSelected Features:")
print(selected_columns)

# Display selected dataset
df_selected = df_clean[selected_columns]

print("\nDataset with Selected Features:")
print(df_selected)

# =====
# 3. Exploratory Data Analysis (EDA)
# =====

print("\nDataset Information:")
df_clean.info()

# =====
# Statistical Summary
# =====

print("\nStatistical Summary:")
print(df_clean.describe())

# =====
# Correlation Matrix
# =====

```

```
print("\nCorrelation Matrix:")
print(df_clean.corr())

# =====
# 4. Histogram
# =====

df_clean.hist(
    figsize=(8, 6),
    color='red',
    edgecolor='black'
)

plt.suptitle("Histogram of Features")
plt.tight_layout()
plt.show()

# =====
# 5. Boxplot
# =====

plt.figure(figsize=(8, 5))

df_clean.boxplot(
    color='red'
)

plt.title("Boxplot of Dataset")
plt.ylabel("Values")
plt.grid(False)

plt.show()
```

Original Dataset:

	Age	Marks	Attendance	Constant
0	20	75	85	1
1	21	80	88	1
2	22	85	90	1
3	23	90	92	1
4	24	95	94	1
5	100	20	50	1

Q1: 21.25

Q3: 23.75

IQR: 2.5

Lower Limit: 17.5

Upper Limit: 27.5

Dataset after Noise Elimination:

	Age	Marks	Attendance	Constant
0	20	75	85	1
1	21	80	88	1
2	22	85	90	1
3	23	90	92	1
4	24	95	94	1

Selected Features:

Index(['Age', 'Marks', 'Attendance'], dtype='object')

Dataset with Selected Features:

	Age	Marks	Attendance
0	20	75	85
1	21	80	88
2	22	85	90
3	23	90	92
4	24	95	94

Dataset Information:

<class 'pandas.core.frame.DataFrame'>

Index: 5 entries, 0 to 4

Data columns (total 4 columns):

#	Column	Non-Null Count	Dtype
0	Age	5 non-null	int64
1	Marks	5 non-null	int64
2	Attendance	5 non-null	int64
3	Constant	5 non-null	int64

dtypes: int64(4)

memory usage: 200.0 bytes

Statistical Summary:

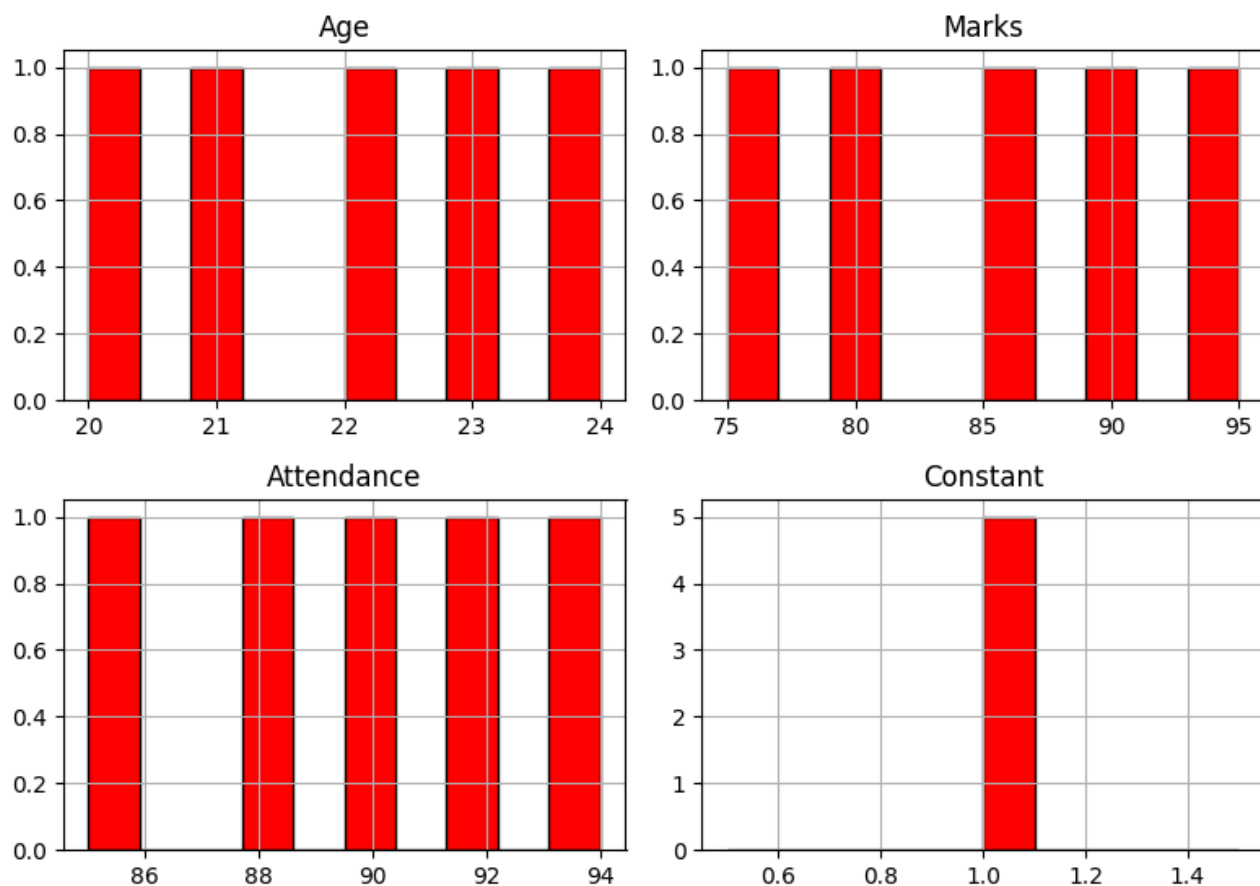
	Age	Marks	Attendance	Constant
count	5.000000	5.000000	5.000000	5.0
mean	22.000000	85.000000	89.800000	1.0
std	1.581139	7.905694	3.49285	0.0
min	20.000000	75.000000	85.000000	1.0
25%	21.000000	80.000000	88.000000	1.0
50%	22.000000	85.000000	90.000000	1.0

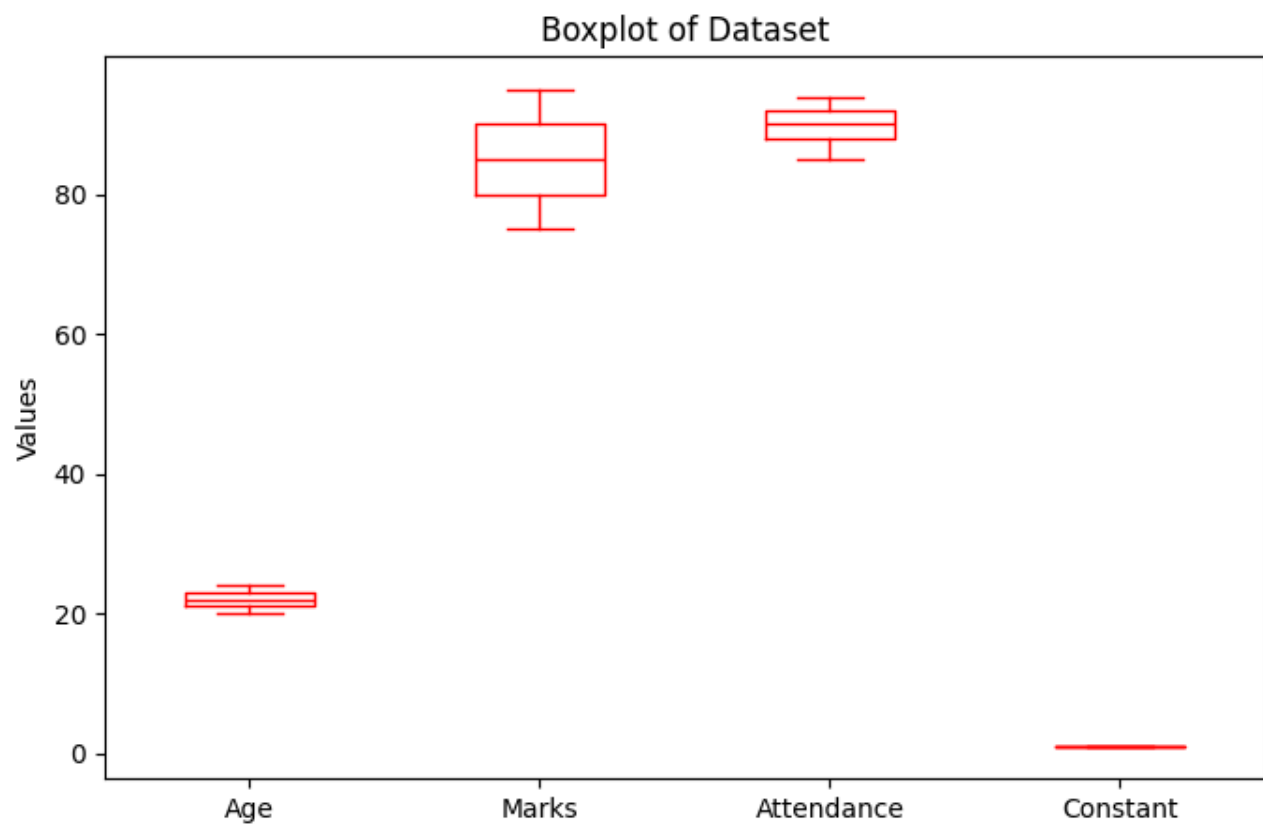
75%	23.000000	90.000000	92.00000	1.0
max	24.000000	95.000000	94.00000	1.0

Correlation Matrix:

	Age	Marks	Attendance	Constant
Age	1.000000	1.000000	0.995893	NaN
Marks	1.000000	1.000000	0.995893	NaN
Attendance	0.995893	0.995893	1.000000	NaN
Constant	NaN	NaN	NaN	NaN

Histogram of Features





Name:- Tanmay Santosh Sanap

Roll No:-40

Student Id:- 5144978

College:- B K BIRLA COLLEGE

Subject:-Data Engineering (Practical No :- 05)

Extracting Data from APIs and Flat Files Using Python1. ObjectiveWrite a Python script to fetch user data from a public REST API, clean it, and merge it with a flat CSV file containing location details.2. Code ImplementationCreate a script named data_extraction.

```
In [1]: import pandas as pd
import requests

# =====
# 1. Extract Data from REST API
# =====

def extract_api_data(url):
    """Fetches data from a REST API and returns a DataFrame."""

    try:
        response = requests.get(url, timeout=10)
        response.raise_for_status()

        data = response.json()

        # Convert JSON data into a DataFrame
        return pd.json_normalize(data)

    except requests.exceptions.RequestException as e:
        print(f"API Error: {e}")
        return pd.DataFrame()

# =====
# 2. Extract Data from CSV File
# =====

def extract_csv_data(file_path):
    """Reads a CSV file."""

    try:
        return pd.read_csv(file_path)
```

```

except FileNotFoundError as e:
    print(f"File Error: {e}")
    return pd.DataFrame()

# =====
# MAIN PROGRAM
# =====

if __name__ == "__main__":

    # -----
    # 1. API Extraction
    # -----

    # JSONPlaceholder mock users API
    api_url = "https://jsonplaceholder.typicode.com/users"

    api_df = extract_api_data(api_url)

    if not api_df.empty:

        api_df = api_df[
            ["id", "name", "email", "company.name"]
        ]

        api_df.rename(
            columns={"company.name": "company"},
            inplace=True
        )

        print("\nAPI Data:")
        print(api_df.head())

    # -----
    # 2. Create and Extract CSV Data
    # -----

    csv_file = "locations.csv"

    # Create sample location data
    location_data = pd.DataFrame(
        {
            "id": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
            "city": [
                "New York",
                "London",
                "Paris",
                "Tokyo",
                "Berlin",
                "Delhi",
                "Sydney",
            ]
        }
    )

```

```

        "Moscow",
        "Cairo",
        "Beijing"
    ]
}
)

# Save DataFrame as CSV
location_data.to_csv(
    csv_file,
    index=False
)

# Read CSV file
csv_df = extract_csv_data(csv_file)

print("\nCSV Data:")
print(csv_df)

# -----
# 3. Data Transformation & Merging
# -----

if not api_df.empty and not csv_df.empty:

    merged_df = pd.merge(
        api_df,
        csv_df,
        on="id",
        how="inner"
    )

    print("\n=====")
    print("Merged ETL Pipeline Data View")
    print("=====")

    print(merged_df)

# -----
# 4. Save Final Data
# -----

output_file = "cleaned_warehouse_profiles.csv"

merged_df.to_csv(
    output_file,
    index=False
)

print(
    f"\nData successfully saved to '{output_file}'"
)

```

API Data:

	id	name	email	company
0	1	Leanne Graham	Sincere@april.biz	Romaguera-Crona
1	2	Ervin Howell	Shanna@melissa.tv	Deckow-Crist
2	3	Clementine Bauch	Nathan@yesenia.net	Romaguera-Jacobson
3	4	Patricia Lebsack	Julianne.OConner@kory.org	Robel-Corkery
4	5	Chelsey Dietrich	Lucio_Hettinger@annie.ca	Keebler LLC

CSV Data:

	id	city
0	1	New York
1	2	London
2	3	Paris
3	4	Tokyo
4	5	Berlin
5	6	Delhi
6	7	Sydney
7	8	Moscow
8	9	Cairo
9	10	Beijing

=====

Merged ETL Pipeline Data View

=====

	id	name	email \
0	1	Leanne Graham	Sincere@april.biz
1	2	Ervin Howell	Shanna@melissa.tv
2	3	Clementine Bauch	Nathan@yesenia.net
3	4	Patricia Lebsack	Julianne.OConner@kory.org
4	5	Chelsey Dietrich	Lucio_Hettinger@annie.ca
5	6	Mrs. Dennis Schulist	Karley_Dach@jasper.info
6	7	Kurtis Weissnat	Telly.Hoeger@billy.biz
7	8	Nicholas Runolfsdottir V	Sherwood@rosamond.me
8	9	Glenna Reichert	Chaim_McDermott@dana.io
9	10	Clementina DuBuque	Rey.Padberg@karina.biz

	company	city
0	Romaguera-Crona	New York
1	Deckow-Crist	London
2	Romaguera-Jacobson	Paris
3	Robel-Corkery	Tokyo
4	Keebler LLC	Berlin
5	Considine-Lockman	Delhi
6	Johns Group	Sydney
7	Abernathy Group	Moscow
8	Yost and Sons	Cairo
9	Hoeger LLC	Beijing

Data successfully saved to 'cleaned_warehouse_profiles.csv'

Name:- Tanmay Santosh Sanap

Roll No:-40

Student Id:- 5144978

College:- B K BIRLA COLLEGE

Subject:-Data Engineering (Practical No :- 06)

To design and configure a scheduled Apache Airflow DAG for orchestrating an ETL workflow through sequential tasks for pipeline initialization, data extraction, and successful pipeline completion.

```
In [1]: # =====
# APACHE AIRFLOW ETL PIPELINE PRACTICAL
# =====

# 1. Install Apache Airflow
!pip install apache-airflow -q

# =====
# 2. Import Libraries
# =====

import os
import subprocess

# =====
# 3. Configure Airflow
# =====

os.environ["AIRFLOW_HOME"] = "/content/airflow"
os.environ["AIRFLOW__CORE__DAGS_FOLDER"] = "/content/airflow/dags"

os.makedirs("/content/airflow/dags", exist_ok=True)

print("Airflow environment configured successfully.")

# =====
# 4. Create Data Extraction Python Script
# =====

with open("/content/airflow/data_extraction.py", "w") as f:
    f.write("""
print("Starting data extraction...")
print("Extracting data from source...")
```

```

print("Data extraction completed successfully.")
"""

print("Data extraction script created successfully.")

# =====
# 5. Create Airflow DAG
# =====

with open("/content/airflow/dags/data_pipeline_dag.py", "w") as f:
    f.write("""
from datetime import datetime, timedelta

from airflow.sdk import DAG
from airflow.providers.standard.operators.bash import BashOperator
from airflow.providers.standard.operators.empty import EmptyOperator

# Default arguments
default_args = {
    "owner": "data_engineering_lab",
    "depends_on_past": False,
    "start_date": datetime(2026, 1, 1),
    "retries": 1,
    "retry_delay": timedelta(minutes=5),
}

# Create DAG
with DAG(
    dag_id="university_etl_orchestration",
    default_args=default_args,
    description="ETL pipeline orchestration",
    schedule="@daily",
    catchup=False,
) as dag:

    # Start task
    start_pipeline = EmptyOperator(
        task_id="start_pipeline"
    )

    # Extraction task
    execute_extraction = BashOperator(
        task_id="run_extraction_script",
        bash_command="python /content/airflow/data_extraction.py"
    )

    # Completion task
    pipeline_complete = BashOperator(
        task_id="log_pipeline_success",
        bash_command='echo "ETL Execution completed successfully"'
    )

```

```

        # Task dependency
        start_pipeline >> execute_extraction >> pipeline_complete
    """

print("DAG created successfully.")
print("DAG Name: university_etl_orchestration")
print("Schedule: Daily")

# =====
# 6. Initialize Airflow Database
# =====

print("\nInitializing Airflow database...\n")

db_result = subprocess.run(
    ["airflow", "db", "migrate"],
    env=os.environ,
    capture_output=True,
    text=True
)

print(db_result.stdout)

if db_result.stderr:
    print(db_result.stderr)

# =====
# 7. Check DAG
# =====

print("\nChecking Airflow DAG...\n")

dag_result = subprocess.run(
    [
        "airflow",
        "dags",
        "list"
    ],
    env=os.environ,
    capture_output=True,
    text=True
)

print(dag_result.stdout)

if dag_result.stderr:
    print(dag_result.stderr)

# =====
# 8. Test Complete DAG

```

```

# =====

print("\n=====")
print("Running DAG tasks...")
print("=====\\n")

result = subprocess.run(
    [
        "airflow",
        "dags",
        "test",
        "university_etl_orchestration",
        "2026-08-20"
    ],
    env=os.environ,
    capture_output=True,
    text=True
)

print(result.stdout)

if result.stderr:
    print(result.stderr)

# =====
# 9. Practical Completed
# =====

print("\n=====")
print("ETL Pipeline Practical Completed!")
print("=====")

```

0:00	6.9/6.9 MB	19.8 MB/s	eta 0:0
0:00:00	607.5/607.5 kB	20.1 MB/s	eta
0:00:00	268.7/268.7 kB	17.5 MB/s	eta
0:00	49.0/49.0 kB	3.0 MB/s	eta 0:0
0:00	98.7/98.7 kB	6.6 MB/s	eta 0:0
0:00:00	163.5/163.5 kB	12.1 MB/s	eta
0:00	43.3/43.3 kB	3.0 MB/s	eta 0:0
0:00	61.2/61.2 kB	4.3 MB/s	eta 0:0
0:00	76.0/76.0 kB	5.7 MB/s	eta 0:0
0:00	46.7/46.7 kB	2.2 MB/s	eta 0:0
0:00:00	117.5/117.5 kB	6.7 MB/s	eta
0:00	71.5/71.5 kB	3.4 MB/s	eta 0:0
0:00	2.3/2.3 MB	26.3 MB/s	eta 0:0
0:00:00	230.5/230.5 kB	12.4 MB/s	eta
0:00	72.5/72.5 kB	3.5 MB/s	eta 0:0
0:00	57.3/57.3 kB	2.0 MB/s	eta 0:0
0:00:00	348.9/348.9 kB	13.4 MB/s	eta
0:00:00	155.6/155.6 kB	8.2 MB/s	eta
0:00	73.8/73.8 kB	4.2 MB/s	eta 0:0
0:00	83.5/83.5 kB	6.0 MB/s	eta 0:0
0:00:00	623.8/623.8 kB	29.4 MB/s	eta
0:00	79.5/79.5 kB	4.2 MB/s	eta 0:0
0:00	69.4/69.4 kB	5.0 MB/s	eta 0:0
0:00:00	778.7/778.7 kB	25.7 MB/s	eta
0:00:00	331.1/331.1 kB	18.9 MB/s	eta
0:00:00	140.1/140.1 kB	9.3 MB/s	eta
0:00	60.3/60.3 kB	5.1 MB/s	eta 0:0
	206.3/206.3 kB	16.4 MB/s	eta

0:00:00

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.

google-adk 2.7.1 requires opentelemetry-api<=1.42.1,>=1.39, but you have opentelemetry-api 1.45.1 which is incompatible.

google-adk 2.7.1 requires opentelemetry-sdk<=1.42.1,>=1.39, but you have opentelemetry-sdk 1.45.1 which is incompatible.

Airflow environment configured successfully.

Data extraction script created successfully.

DAG created successfully.

DAG Name: university_etl_orchestration

Schedule: Daily

Initializing Airflow database...

2026-10-07T19:22:26.135074Z [info] Performing upgrade to the metadata database [airflow.cli.commands.db_command] loc=db_command.py:134 url=sqlite:///content/airflow/airflow.db

2026-10-07T19:22:26.916731Z [info] Context impl SQLiteImpl. [alembic.runtime.migration] loc=migration.py:205

2026-10-07T19:22:26.917023Z [info] Will assume non-transactional DDL. [alembic.runtime.migration] loc=migration.py:208

2026-10-07T19:22:28.141525Z [info] Context impl SQLiteImpl. [alembic.runtime.migration] loc=migration.py:205

2026-10-07T19:22:28.141765Z [info] Will assume non-transactional DDL. [alembic.runtime.migration] loc=migration.py:208

2026-10-07T19:22:28.142374Z [info] Creating Airflow database tables from the ORM [airflow.utils.db] loc=db.py:749

2026-10-07T19:22:28.142578Z [info] Creating global lock context [airflow.utils.db] loc=db.py:730

2026-10-07T19:22:28.142769Z [info] Pool status: Pool size: 5 Connections in pool: 0 Current Overflow: -4 Current Checked out connections: 1 [airflow.utils.db] loc=db.py:733

2026-10-07T19:22:28.142932Z [info] Creating metadata [airflow.utils.db] loc=db.py:735

2026-10-07T19:22:28.844274Z [info] Getting alembic config [airflow.utils.db] loc=db.py:738

2026-10-07T19:22:28.846307Z [info] Stamping migration head [airflow.utils.db] loc=db.py:741

2026-10-07T19:22:28.848210Z [info] Context impl SQLiteImpl. [alembic.runtime.migration] loc=migration.py:205

2026-10-07T19:22:28.848457Z [info] Will assume non-transactional DDL. [alembic.runtime.migration] loc=migration.py:208

2026-10-07T19:22:28.899360Z [info] Running stamp_revision -> d2f4e1b3c5a7 [alembic.runtime.migration] loc=migration.py:611

2026-10-07T19:22:28.905537Z [info] Airflow database tables created [airflow.utils.db] loc=db.py:744

2026-10-07T19:22:29.110352Z [info] Database migration done! [airflow.cli.commands.db_command] loc=db_command.py:152

Checking Airflow DAG...

No data found

=====
Running DAG tasks...
=====

2026-10-07T19:22:40.824843Z [info] DAG bundles loaded: dags-folder, example_dags, apache-airflow-providers-common-sql-example-dags, apache-airflow-providers-standard-example-dags [airflow.dag_processing.bundles.manager.DagBundlesManager] loc=manager.py:304
2026-10-07T19:22:41.025060Z [info] Added new DAG bundle dags-folder to the database [airflow.dag_processing.bundles.manager.DagBundlesManager] loc=manager.py:378
2026-10-07T19:22:41.025568Z [info] Added new DAG bundle example_dags to the database [airflow.dag_processing.bundles.manager.DagBundlesManager] loc=manager.py:378
2026-10-07T19:22:41.025919Z [info] Added new DAG bundle apache-airflow-providers-common-sql-example-dags to the database [airflow.dag_processing.bundles.manager.DagBundlesManager] loc=manager.py:378
2026-10-07T19:22:41.026223Z [info] Added new DAG bundle apache-airflow-providers-standard-example-dags to the database [airflow.dag_processing.bundles.manager.DagBundlesManager] loc=manager.py:378
2026-10-07T19:22:41.033623Z [info] Filling up the DagBag from /content/airflow/dags [airflow.dag_processing.dagbag.BundleDagBag] loc=dagbag.py:460
2026-10-07T19:22:41.055919Z [info] Bulk-writing dags to db [airflow.serialization.definitions.dag] count=1 loc=dag.py:218
2026-10-07T19:22:41.069298Z [info] Creating ORM DAG for university_etl_orchestration [airflow.dag_processing.collection] loc=collection.py:102
2026-10-07T19:22:41.086183Z [info] get next_dagrun_info_v2 [airflow.serialization.definitions.dag] last_automated_run_info=None loc=dag.py:453 next_info=DagRunInfo(run_after=DateTime(2026, 10, 7, 0, 0, 0, tzinfo=Timezone('UTC')), data_interval=DataInterval(start=DateTime(2026, 10, 7, 0, 0, 0, tzinfo=Timezone('UTC')), end=DateTime(2026, 10, 7, 0, 0, 0, tzinfo=Timezone('UTC'))), partition_date=None, partition_key=None)
2026-10-07T19:22:41.086525Z [info] setting next dagrun info [airflow.models.dag] loc=dag.py:848 next_dagrun='2026-10-07 00:00:00+00:00' next_dagrun_create_after='2026-10-07 00:00:00+00:00' next_dagrun_data_interval_end='2026-10-07 00:00:00+00:00' next_dagrun_data_interval_start='2026-10-07 00:00:00+00:00' next_dagrun_partition_date=None next_dagrun_partition_key=None
2026-10-07T19:22:41.145485Z [info] DAG bundles loaded: dags-folder, example_dags, apache-airflow-providers-common-sql-example-dags, apache-airflow-providers-standard-example-dags [airflow.dag_processing.bundles.manager.DagBundlesManager] loc=manager.py:304
2026-10-07T19:22:41.152454Z [info] Filling up the DagBag from /content/airflow/dags [airflow.dag_processing.dagbag.BundleDagBag] loc=dagbag.py:460
2026-10-07T19:22:41.158351Z [info] Bulk-writing dags to db [airflow.serialization.definitions.dag] count=1 loc=dag.py:218
2026-10-07T19:22:41.173434Z [info] get next_dagrun_info_v2 [airflow.serialization.definitions.dag] last_automated_run_info=None loc=dag.py:453 next_info=DagRunInfo(run_after=DateTime(2026, 10, 7, 0, 0, 0, tzinfo=Timezone('UTC')), data_interval=DataInterval(start=DateTime(2026, 10, 7, 0, 0, 0, tzinfo=Timezone('UTC')), end=DateTime(2026, 10, 7, 0, 0, 0, tzinfo=Timezone('UTC'))), partition_date=None, partition_key=None)

```

0, 7, 0, 0, 0, tzinfo=Timezone('UTC')), end=DateTime(2026, 10, 7, 0, 0,
0, tzinfo=Timezone('UTC'))), partition_date=None, partition_key=None)
2026-10-07T19:22:41.173727Z [info      ] setting next dagrun info      [a
irflow.models.dag] loc=dag.py:848 next_dagrun='2026-10-07 00:00:00+00:00'
next_dagrun_create_after='2026-10-07 00:00:00+00:00' next_dagrun_data_int
erval_end='2026-10-07 00:00:00+00:00' next_dagrun_data_interval_start='20
26-10-07 00:00:00+00:00' next_dagrun_partition_date=None next_dagrun_part
ition_key=None
2026-10-07T19:22:41.197054Z [info      ] creating dag run              [a
irflow.serialization.definitions.dag] loc=dag.py:615 logical_date=DateTim
e(2026, 8, 20, 0, 0, 0, tzinfo=Timezone('UTC')) partition_key=None run_af
ter=DateTime(2026, 10, 7, 19, 22, 41, 144247, tzinfo=Timezone('UTC')) ru
n_id>manual__2026-10-07T19:22:41.144247+00:00
2026-10-07T19:22:41.276123Z [info      ] Created dag run.              [a
irflow.models.dagrun] dagrun=<DagRun university_etl_orchestration @ 202
6-08-20 00:00:00+00:00: manual__2026-10-07T19:22:41.144247+00:00, state:r
unning, queued_at: None. run_type: manual> loc=dagrun.py:2464
2026-10-07T19:22:41.303186Z [info      ] [DAG TEST] starting task_id=star
t_pipeline map_index=-1 [airflow.sdk.definitions.dag] loc=dag.py:1501
2026-10-07T19:22:41.303673Z [info      ] [DAG TEST] running task <TaskInst
ance: university_etl_orchestration.start_pipeline manual__2026-10-07T19:2
2:41.144247+00:00 [TaskInstanceState.SCHEDULED] ti_id=01a117d1-64bc-7c7
1-9751-5859c3ef159b> [airflow.sdk.definitions.dag] loc=dag.py:1504
2026-10-07T19:22:48.672591Z [info      ] Task started                  [a
irflow.api_fastapi.execution_api.routes.task_instances] correlation_id=01
a117d1-6b5d-7509-9978-01e31e982826 hostname=07db903bcc43 loc=task_instanc
es.py:226 previous_state=queued ti_id=01a117d1-64bc-7c71-9751-5859c3ef159
b
2026-10-07T19:22:48.675374Z [info      ] Task instance state updated    [a
irflow.api_fastapi.execution_api.routes.task_instances] correlation_id=01
a117d1-6b5d-7509-9978-01e31e982826 loc=task_instances.py:254 rows_affected=1 ti_id=01a117d1-64bc-7c71-9751-5859c3ef159b
Task instance is in running state
Previous state of the Task instance: TaskInstanceState.QUEUED
Current task name:start_pipeline
Dag name:university_etl_orchestration
2026-10-07T19:22:48.702655Z [info      ] ::endgroup::                  [t
ask] loc=task_runner.py:2232
2026-10-07T19:22:48.702903Z [info      ] ::group::Post Execute          [t
ask] loc=task_runner.py:1580
2026-10-07T19:22:48.730506Z [info      ] Task instance state updated    [a
irflow.api_fastapi.execution_api.routes.task_instances] correlation_id=01
a117d1-81ff-789b-888c-ee76e20456a9 loc=task_instances.py:485 new_state=su
ccess rows_affected=1 ti_id=01a117d1-64bc-7c71-9751-5859c3ef159b
Task instance in success state
Previous state of the Task instance: TaskInstanceState.RUNNING
Task operator:<Task(EmptyOperator): start_pipeline>
2026-10-07T19:22:48.740209Z [info      ] ::endgroup::                  [t
ask] loc=task_runner.py:2379
2026-10-07T19:22:48.756725Z [info      ] [DAG TEST] end task task_id=star
t_pipeline map_index=-1 [airflow.sdk.definitions.dag] loc=dag.py:1563
2026-10-07T19:22:48.770895Z [info      ] [DAG TEST] starting task_id=run_e
xtraction_script map_index=-1 [airflow.sdk.definitions.dag] loc=dag.py:15
01
2026-10-07T19:22:48.771618Z [info      ] [DAG TEST] running task <TaskInst

```

```

ance: university_etl_orchestration.run_extraction_script manual__2026-1
0-07T19:22:41.144247+00:00 [TaskInstanceState.SCHEDULED] ti_id=01a117d1-6
4bd-7733-82ae-8b65a276ca2d> [airflow.sdk.definitions.dag] loc=dag.py:1504
2026-10-07T19:22:48.791738Z [info      ] Task started [a
irflow.api_fastapi.execution_api.routes.task_instances] correlation_id=01
a117d1-824b-78bb-ad21-263ecbfleaca hostname=07db903bcc43 loc=task_instanc
es.py:226 previous_state=queued ti_id=01a117d1-64bd-7733-82ae-8b65a276ca2
d
2026-10-07T19:22:48.793207Z [info      ] Task instance state updated [a
irflow.api_fastapi.execution_api.routes.task_instances] correlation_id=01
a117d1-824b-78bb-ad21-263ecbfleaca loc=task_instances.py:254 rows_affected=1 ti_id=01a117d1-64bd-7733-82ae-8b65a276ca2d
2026-10-07T19:22:48.825836Z [info      ] Updating RenderedTaskInstanceFields [airflow.api_fastapi.execution_api.routes.task_instances] correlation_id=01a117d1-826f-72df-84de-342bb9c1913c field_count=3 loc=task_instances.py:1017 ti_id=01a117d1-64bd-7733-82ae-8b65a276ca2d
Task instance is in running state
Previous state of the Task instance: TaskInstanceState.QUEUED
Current task name:run_extraction_script
Dag name:university_etl_orchestration
2026-10-07T19:22:48.846788Z [info      ] ::endgroup:: [t
ask] loc=task_runner.py:2232
2026-10-07T19:22:48.847566Z [info      ] Tmp dir root location: /tmp [a
irflow.task.hooks.airflow.providers.standard.hooks.subprocess.SubprocessHook] loc=subprocess.py:78
2026-10-07T19:22:48.848045Z [info      ] Running command: ['/usr/bin/bash', '-c', 'python /content/airflow/data_extraction.py'] [airflow.task.hooks.airflow.providers.standard.hooks.subprocess.SubprocessHook] loc=subprocess.py:80
2026-10-07T19:22:48.848712Z [info      ] Output: [a
irflow.task.hooks.airflow.providers.standard.hooks.subprocess.SubprocessHook] loc=subprocess.py:92
2026-10-07T19:22:48.882089Z [info      ] Starting data extraction... [a
irflow.task.hooks.airflow.providers.standard.hooks.subprocess.SubprocessHook] loc=subprocess.py:99
2026-10-07T19:22:48.882314Z [info      ] Extracting data from source... [a
irflow.task.hooks.airflow.providers.standard.hooks.subprocess.SubprocessHook] loc=subprocess.py:99
2026-10-07T19:22:48.882422Z [info      ] Data extraction completed successfully. [airflow.task.hooks.airflow.providers.standard.hooks.subprocess.SubprocessHook] loc=subprocess.py:99
2026-10-07T19:22:48.886950Z [info      ] Command exited with return code 0 [airflow.task.hooks.airflow.providers.standard.hooks.subprocess.SubprocessHook] loc=subprocess.py:103
2026-10-07T19:22:48.887396Z [info      ] ::group::Post Execute [t
ask] loc=task_runner.py:1580
2026-10-07T19:22:48.887632Z [info      ] Pushing xcom [t
ask] loc=task_runner.py:2280 ti=RuntimeTaskInstance(id=UUID('01a117d1-64bd-7733-82ae-8b65a276ca2d'), task_id='run_extraction_script', dag_id='university_etl_orchestration', run_id='manual__2026-10-07T19:22:41.144247+00:00', try_number=1, dag_version_id=UUID('01a117d1-645a-79a9-b9cb-98a04ae127b4'), map_index=-1, hostname='07db903bcc43', context_carrier=None, queue='default', task=<Task(BashOperator): run_extraction_script>, max_tries=1, start_date=datetime.datetime(2026, 10, 7, 19, 22, 48, 779200, tzinfo=datetime.timezone.utc), end_date=None, state=<TaskInstanceState.RUNNING

```

```

G: 'running'>, is_mapped=False, rendered_map_index=None, sentry_integrati
on='')
2026-10-07T19:22:48.922495Z [info      ] Task instance state updated    [a
irflow.api_fastapi.execution_api.routes.task_instances] correlation_id=01
a117d1-82cc-7ad1-9320-bfa9fe2ecfd1 loc=task_instances.py:485 new_state=su
ccess rows_affected=1 ti_id=01a117d1-64bd-7733-82ae-8b65a276ca2d
Task instance in success state
Previous state of the Task instance: TaskInstanceState.RUNNING
Task operator:<Task(BashOperator): run_extraction_script>
2026-10-07T19:22:48.931203Z [info      ] ::endgroup::                      [t
ask] loc=task_runner.py:2379
2026-10-07T19:22:48.940826Z [info      ] [DAG TEST] end task task_id=run_e
xtraction_script map_index=-1 [airflow.sdk.definitions.dag] loc=dag.py:15
63
2026-10-07T19:22:48.951733Z [info      ] [DAG TEST] starting task_id=log_p
ipeline_success map_index=-1 [airflow.sdk.definitions.dag] loc=dag.py:150
1
2026-10-07T19:22:48.951986Z [info      ] [DAG TEST] running task <TaskInst
ance: university_etl_orchestration.log_pipeline_success manual__2026-10-0
7T19:22:41.144247+00:00 [TaskInstanceState.SCHEDULED] ti_id=01a117d1-64b
e-7f7e-8d4f-5b4730885687> [airflow.sdk.definitions.dag] loc=dag.py:1504
2026-10-07T19:22:48.970313Z [info      ] Task started                      [a
irflow.api_fastapi.execution_api.routes.task_instances] correlation_id=01
a117d1-82fe-7134-8c28-74ac306bd375 hostname=07db903bcc43 loc=task_instanc
es.py:226 previous_state=queued ti_id=01a117d1-64be-7f7e-8d4f-5b473088568
7
2026-10-07T19:22:48.971683Z [info      ] Task instance state updated    [a
irflow.api_fastapi.execution_api.routes.task_instances] correlation_id=01
a117d1-82fe-7134-8c28-74ac306bd375 loc=task_instances.py:254 rows_affected=1 ti_id=01a117d1-64be-7f7e-8d4f-5b4730885687
2026-10-07T19:22:48.994963Z [info      ] Updating RenderedTaskInstanceFiel
ds [airflow.api_fastapi.execution_api.routes.task_instances] correlatio
n_id=01a117d1-831b-71ca-b268-c1e130906c17 field_count=3 loc=task_instanc
es.py:1017 ti_id=01a117d1-64be-7f7e-8d4f-5b4730885687
Task instance is in running state
Previous state of the Task instance: TaskInstanceState.QUEUED
Current task name:log_pipeline_success
Dag name:university_etl_orchestration
2026-10-07T19:22:49.007511Z [info      ] ::endgroup::                      [t
ask] loc=task_runner.py:2232
2026-10-07T19:22:49.008217Z [info      ] Tmp dir root location: /tmp      [a
irflow.task.hooks.airflow.providers.standard.hooks.subprocess.SubprocessH
ook] loc=subprocess.py:78
2026-10-07T19:22:49.008614Z [info      ] Running command: ['/usr/bin/bas
h', '-c', 'echo "ETL Execution completed successfully"'] [airflow.task.ho
oks.airflow.providers.standard.hooks.subprocess.SubprocessHook] loc=subpr
ocess.py:80
2026-10-07T19:22:49.009261Z [info      ] Output:                          [a
irflow.task.hooks.airflow.providers.standard.hooks.subprocess.SubprocessH
ook] loc=subprocess.py:92
2026-10-07T19:22:49.010611Z [info      ] ETL Execution completed successfu
lly [airflow.task.hooks.airflow.providers.standard.hooks.subprocess.Subpr
ocessHook] loc=subprocess.py:99
2026-10-07T19:22:49.010847Z [info      ] Command exited with return code 0
[airflow.task.hooks.airflow.providers.standard.hooks.subprocess.Subproces

```

```

sHook] loc=subprocess.py:103
2026-10-07T19:22:49.011243Z [info      ] ::group::Post Execute      [t
ask] loc=task_runner.py:1580
2026-10-07T19:22:49.011437Z [info      ] Pushing xcom                [t
ask] loc=task_runner.py:2280 ti=RuntimeTaskInstance(id=UUID('01a117d1-64be-7f7e-8d4f-5b4730885687'), task_id='log_pipeline_success', dag_id='university_etl_orchestration', run_id='manual__2026-10-07T19:22:41.144247+00:00', try_number=1, dag_version_id=UUID('01a117d1-645a-79a9-b9cb-98a04ae127b4'), map_index=-1, hostname='07db903bcc43', context_carrier=None, queue='default', task=<Task(BashOperator): log_pipeline_success>, max_tries=1, start_date=datetime.datetime(2026, 10, 7, 19, 22, 48, 958345, tzinfo=datetime.timezone.utc), end_date=None, state=<TaskInstanceState.RUNNING: 'running'>, is_mapped=False, rendered_map_index=None, sentry_integration='')
2026-10-07T19:22:49.042133Z [info      ] Task instance state updated [airflow.api_fastapi.execution_api.routes.task_instances] correlation_id=01a117d1-8345-728b-bf56-81ac6511eb78 loc=task_instances.py:485 new_state=success rows_affected=1 ti_id=01a117d1-64be-7f7e-8d4f-5b4730885687
Task instance in success state
Previous state of the Task instance: TaskInstanceState.RUNNING
Task operator:<Task(BashOperator): log_pipeline_success>
2026-10-07T19:22:49.051534Z [info      ] ::endgroup::                [t
ask] loc=task_runner.py:2379
2026-10-07T19:22:49.061200Z [info      ] [DAG TEST] end task task_id=log_pipeline_success map_index=-1 [airflow.sdk.definitions.dag] loc=dag.py:1563
2026-10-07T19:22:49.063851Z [info      ] Marking run <DagRun university_etl_orchestration @ 2026-08-20 00:00:00+00:00: manual__2026-10-07T19:22:41.144247+00:00, state:running, queued_at: None. run_type: manual> successful [airflow.models.dagrun.DagRun] loc=dagrun.py:1249
Dag run in success state
Dag run start:2026-08-20 00:00:00+00:00 end:2026-10-07 19:22:49.064041+00:00
2026-10-07T19:22:49.064929Z [info      ] DagRun Finished: dag_id=university_etl_orchestration, logical_date=2026-08-20 00:00:00+00:00, run_id=manual__2026-10-07T19:22:41.144247+00:00, run_start_date=2026-08-20 00:00:00+00:00, run_end_date=2026-10-07 19:22:49.064041+00:00, run_duration=421696.9.064041, state=success, run_type=manual, data_interval_start=2026-08-20 00:00:00+00:00, data_interval_end=2026-08-20 00:00:00+00:00, [airflow.models.dagrun.DagRun] loc=dagrun.py:1315

```

```

=====
ETL Pipeline Practical Completed!
=====

```

Name:- Tanmay Santosh Sanap

Roll No:-40

Student Id:- 5144978

College:- B K BIRLA COLLEGE

Subject:-Data Engineering (Practical No :- 07)

To design and implement an ETL pipeline using Python for extracting data from CSV files, cleaning and transforming the data, validating the records, and loading the processed data into a database.

```
In [3]: import pandas as pd
import sqlite3

# =====
# 1. CSV ETL
# =====

print("\n--- 1. CSV ETL Pipeline ---")

df = pd.DataFrame({
    "ID": [1, 2, 3],
    "Name": ["Amit", "Neha", "Rahul"],
    "Age": [22, 25, 29],
    "Salary": [30000, 40000, 35000]
})

# Data Cleaning
df["Age"] = df["Age"].fillna(df["Age"].median())

# Data Transformation
df["Salary"] = df["Salary"] * 1.10

# Store in SQLite database
con = sqlite3.connect("etl.db")

df.to_sql(
    "employees",
    con,
    if_exists="replace",
    index=False
)

con.close()

print(df)
```

```

# =====
# 2. MULTIPLE CSV FILES
# =====

print("\n--- 2. Combining Multiple CSV Files ---")

a = pd.DataFrame({
    "ID": [1, 2],
    "Name": ["Amit", "Neha"]
})

b = pd.DataFrame({
    "ID": [3, 4],
    "Name": ["Rahul", "Priya"]
})

# Combine datasets
combined = pd.concat(
    [a, b],
    ignore_index=True
)

print(combined)

# =====
# 3. JSON ETL
# =====

print("\n--- 3. JSON ETL Pipeline ---")

df = pd.DataFrame({
    "ID": [1, 2, 3],
    "Name": ["Amit", "Neha", "Rahul"],
    "Age": [22, 25, 28]
})

# Transformation
df["Age_Group"] = df["Age"].apply(
    lambda x: "Young" if x < 25 else "Adult"
)

# Store JSON
df.to_json(
    "users.json",
    orient="records"
)

# Store in SQLite
con = sqlite3.connect("users.db")

df.to_sql(
    "users",

```

```

        con,
        if_exists="replace",
        index=False
    )

con.close()

print(df)

# =====
# 4. REMOVE INVALID RECORDS
# =====

print("\n--- 4. Invalid Record Removal ---")

df = pd.DataFrame({
    "ID": [1, 2, 3],
    "Age": [22, -5, 150]
})

print("Original Records:")

print(df)

# Remove invalid age records
df = df[
    (df["Age"] >= 0) &
    (df["Age"] <= 100)
]

print("\nAfter Removing Invalid Records:")

print(df)

# =====
# 5. DATA VALIDATION
# =====

print("\n--- 5. Data Validation ---")

df = pd.DataFrame({
    "ID": [1, 2, 3],
    "Age": [22, 25, 150]
})

# Validate age
valid = df[
    (df["Age"] >= 18) &
    (df["Age"] <= 100)
]

# Store validated records

```

```

con = sqlite3.connect("validated.db")

valid.to_sql(
    "employees",
    con,
    if_exists="replace",
    index=False
)

con.close()

print("Validated Records:")

print(valid)

# =====
# 6. INCREMENTAL LOADING
# =====

print("\n--- 6. Incremental Data Loading ---")

df = pd.DataFrame({
    "ID": [1, 2, 3],
    "Name": ["Amit", "Neha", "Rahul"]
})

con = sqlite3.connect("incremental.db")

# Check whether table already exists
old = pd.read_sql(
    "SELECT name FROM sqlite_master "
    "WHERE type='table'",
    con
)

if old.empty:

    # First load
    df.to_sql(
        "employees",
        con,
        index=False
    )

else:

    # Append new records
    new = df

    new.to_sql(
        "employees",
        con,
        if_exists="append",

```

```
        index=False
    )

print("New Records Loaded:")

print(df)

con.close()

# =====
# FINAL MESSAGE
# =====

print("\n===== ALL ETL OPERATIONS COMPLETED =====")
```

--- 1. CSV ETL Pipeline ---

	ID	Name	Age	Salary
0	1	Amit	22	33000.0
1	2	Neha	25	44000.0
2	3	Rahul	29	38500.0

--- 2. Combining Multiple CSV Files ---

	ID	Name
0	1	Amit
1	2	Neha
2	3	Rahul
3	4	Priya

--- 3. JSON ETL Pipeline ---

	ID	Name	Age	Age_Group
0	1	Amit	22	Young
1	2	Neha	25	Adult
2	3	Rahul	28	Adult

--- 4. Invalid Record Removal ---

Original Records:

	ID	Age
0	1	22
1	2	-5
2	3	150

After Removing Invalid Records:

	ID	Age
0	1	22

--- 5. Data Validation ---

Validated Records:

	ID	Age
0	1	22
1	2	25

--- 6. Incremental Data Loading ---

New Records Loaded:

	ID	Name
0	1	Amit
1	2	Neha
2	3	Rahul

===== ALL ETL OPERATIONS COMPLETED =====

Name:- Tanmay Santosh Sanap

Roll No:-40

Student Id:- 5144978

College:- B K BIRLA COLLEGE

Subject:-Data Engineering (Practical No :- 08)

To implement PySpark DataFrame operations for reading and analyzing CSV data, including schema inspection, filtering, grouping, aggregation, duplicate removal, joining datasets, and calculating average sales by product category.

```
In [1]: # =====
# PRACTICAL 8 - PYSPARK DATA OPERATIONS
# =====

# If PySpark is not installed in Google Colab, run this first:
# !pip install pyspark -q

from pyspark.sql import SparkSession
from pyspark.sql.functions import col, avg, round
import os

print("=" * 60)
print("                                PRACTICAL 8 - PYSPARK")
print("=" * 60)

# =====
# 0. SETUP: INITIALIZE SPARK
# =====

spark = SparkSession.builder \
    .appName("PySpark_Data_Operations") \
    .master("local[*]") \
    .getOrCreate()

# =====
# CREATE SALES CSV FILE
# =====

with open("sales.csv", "w") as f:
    f.write("transaction_id,product_id,quantity,sale_amount\n")
    f.write("1,101,2,50.0\n")
```

```

f.write("2,102,1,120.0\n")
f.write("3,101,4,100.0\n")
f.write("3,101,4,100.0\n")    # Duplicate
f.write("4,103,1,30.0\n")
f.write("5,101,0,0.0\n")    # Invalid quantity

# =====
# CREATE PRODUCTS CSV FILE
# =====

with open("products.csv", "w") as f:

    f.write("product_id,product_name,category\n")
    f.write("101,Wireless Mouse,Electronics\n")
    f.write("102,Mechanical Keyboard,Electronics\n")
    f.write("103,Coffee Mug,Home & Kitchen\n")

# =====
# 1. READ CSV AND DISPLAY SCHEMA
# =====

print("\n--- 1. Reading Data and Displaying Schema ---")

# Read sales data
sales_df = spark.read.csv(
    "sales.csv",
    header=True,
    inferSchema=True
)

# Read products data
products_df = spark.read.csv(
    "products.csv",
    header=True,
    inferSchema=True
)

# Display schema
print("\nSales Data Schema:")

sales_df.printSchema()

# =====
# 2. REMOVE DUPLICATE RECORDS
# =====

print("\n--- 2. Removing Duplicates ---")

print(
    "Original row count:",
    sales_df.count()

```

```

)

# Remove duplicate rows
clean_sales_df = sales_df.dropDuplicates()

print(
    "Row count after dropping duplicates:",
    clean_sales_df.count()
)

print("\nData after removing duplicates:")

clean_sales_df.show()

# =====
# 3. FILTERING OPERATIONS
# =====

print("\n--- 3. Filtering Operations ---")

# Remove records where quantity is 0 or less
filtered_sales_df = clean_sales_df.filter(
    col("quantity") > 0
)

print("\nSales with quantity greater than 0:")

filtered_sales_df.show()

# =====
# 4. JOIN TWO DATASETS
# =====

print("\n--- 4. Joining DataFrames ---")

# Join sales and products using product_id
joined_df = filtered_sales_df.join(
    products_df,
    on="product_id",
    how="inner"
)

print("\nJoined Data:")

joined_df.show()

# =====
# 5. AVERAGE SALES BY PRODUCT CATEGORY
# =====

print("\n--- 5. Average Sales by Product Category ---")

```

```

# Group by category
# Calculate average sale amount
# Round result to 2 decimal places

avg_sales_df = joined_df.groupBy(
    "category"
).agg(
    round(
        avg("sale_amount"),
        2
    ).alias("average_sales")
)

print("\nAverage Sales by Category:")

avg_sales_df.show()

# =====
# CLEANUP
# =====

spark.stop()

os.remove("sales.csv")
os.remove("products.csv")

print("=" * 60)
print("          ALL PRACTICAL 8 TASKS COMPLETED")
print("=" * 60)

```

PRACTICAL 8 - PYSPARK

--- 1. Reading Data and Displaying Schema ---

Sales Data Schema:

```
root
|-- transaction_id: integer (nullable = true)
|-- product_id: integer (nullable = true)
|-- quantity: integer (nullable = true)
|-- sale_amount: double (nullable = true)
```

--- 2. Removing Duplicates ---

Original row count: 6

Row count after dropping duplicates: 5

Data after removing duplicates:

transaction_id	product_id	quantity	sale_amount
1	101	2	50.0
4	103	1	30.0
2	102	1	120.0
3	101	4	100.0
5	101	0	0.0

--- 3. Filtering Operations ---

Sales with quantity greater than 0:

transaction_id	product_id	quantity	sale_amount
1	101	2	50.0
4	103	1	30.0
2	102	1	120.0
3	101	4	100.0

--- 4. Joining DataFrames ---

Joined Data:

product_id	transaction_id	quantity	sale_amount	product_name	category
101	1	2	50.0	Wireless Mouse	Electronics
103	4	1	30.0	Coffee Mug	Home

& Kitchen						
102	2	1	120.0	Mechanical Keyboard	El	
ectronics						
101	3	4	100.0	Wireless Mouse	El	
ectronics						
+-----+	+-----+	+-----+	+-----+	+-----+	+-----+	
+-----+						

--- 5. Average Sales by Product Category ---

Average Sales by Category:

+-----+	+-----+
category	average_sales
+-----+	+-----+
Home & Kitchen	30.0
Electronics	90.0
+-----+	+-----+

```
=====
ALL PRACTICAL 8 TASKS COMPLETED
=====
```

Name:- Tanmay Santosh Sanap

Roll No:-40

Student Id:- 5144978

College:- B K BIRLA COLLEGE

Subject:-Data Engineering (Practical No :- 09)

To design and implement an end-to-end data pipeline that extracts e-commerce data from CSV files, cleans and transforms the data using Python/Pandas, loads it into a SQL database, generates a report, and handles both historical and newly arriving data.

```
In [2]: import pandas as pd
import sqlite3
import os

print("=" * 65)
print("                                PRACTICAL 9 - ETL PIPELINE")
print("=" * 65)

# =====
# TASK 1
# END-TO-END CSV -> PANDAS -> CLEANING -> SQL -> REPORT
# =====

print("\n\n===== TASK 1 =====")
print("End-to-End CSV to SQL Pipeline")

csv_file = "raw_sales.csv"
db_file = "task1_pipeline.db"

# -----
# 1. SETUP: Create messy raw data
# -----

with open(csv_file, "w") as f:

    f.write("transaction_id,date,amount,category\n")

    f.write("1,2026-10-01,150.50,Electronics\n")

    f.write("2,,200.00,Clothing\n")          # Missing date

    f.write("3,2026-10-03,ERROR,Electronics\n") # Invalid amount
```

```

f.write("4,2026-10-04,99.99,Clothing\n")

# -----
# 2. EXTRACT: Read CSV using Pandas
# -----

df = pd.read_csv(csv_file)

print("\nOriginal Data:")
print(df)

# -----
# 3. DATA CLEANING
# -----

# Remove rows where date is missing
df = df.dropna(subset=["date"])

# Convert amount to numeric
# Invalid values become NaN
df["amount"] = pd.to_numeric(
    df["amount"],
    errors="coerce"
)

# Remove rows where amount is invalid
df = df.dropna(subset=["amount"])

# Format date
df["date"] = pd.to_datetime(
    df["date"]
).dt.strftime("%Y-%m-%d")

print("\nCleaned Data:")
print(df)

# -----
# 4. LOAD INTO SQL DATABASE
# -----

conn = sqlite3.connect(db_file)

df.to_sql(
    "cleaned_sales",
    conn,
    if_exists="replace",

```

```

        index=False
    )

# -----
# 5. GENERATE REPORT USING SQL
# -----

report_query = """
SELECT
    category,
    COUNT(transaction_id) AS total_sales,
    SUM(amount) AS total_revenue
FROM cleaned_sales
GROUP BY category
"""

report_df = pd.read_sql(
    report_query,
    conn
)

print("\nFinal Aggregated Report:")
print(
    report_df.to_string(index=False)
)

conn.close()

# =====
# TASK 2
# E-COMMERCE ETL PIPELINE
# =====

print("\n\n===== TASK 2 =====")
print("E-Commerce ETL Pipeline")

db_file = "task2_ecommerce.db"

# -----
# 1. EXTRACT
# Create Customers Data
# -----

customers = pd.DataFrame({
    "cust_id": [1, 2],
    "name": ["Alice", "Bob"]
})

```

```

# -----
# Products Data
# -----

products = pd.DataFrame({
    "prod_id": [101, 102],
    "price": [1200.0, 50.0]
})

# -----
# Orders Data
# -----

orders = pd.DataFrame({
    "order_id": [1001, 1002, 1003],
    "cust_id": [1, 1, 2],
    "prod_id": [101, 102, 102],
    "qty": [1, 2, 1]
})

# -----
# Payments Data
# -----

payments = pd.DataFrame({
    "order_id": [1001, 1002, 1003],
    "status": [
        "Completed",
        "Failed",
        "Completed"
    ]
})

print("\nCustomers:")
print(customers)

print("\nProducts:")
print(products)

print("\nOrders:")
print(orders)

print("\nPayments:")
print(payments)

# -----
# 2. TRANSFORM
# -----

```

```

# Join Orders and Payments
merged = orders.merge(
    payments,
    on="order_id"
)

# Keep only successful payments
successful_orders = merged[
    merged["status"] == "Completed"
]

# Join with Products
successful_orders = successful_orders.merge(
    products,
    on="prod_id"
)

# Calculate total transaction value
successful_orders["total_value"] = (
    successful_orders["qty"] *
    successful_orders["price"]
)

# Join with Customers
final_merged = successful_orders.merge(
    customers,
    on="cust_id"
)

# Calculate customer lifetime revenue
customer_revenue = (
    final_merged
    .groupby("name")["total_value"]
    .sum()
    .reset_index()
)

# Rename column
customer_revenue.rename(
    columns={
        "total_value": "lifetime_revenue"
    },
    inplace=True
)

# -----
# 3. LOAD INTO SQL DATABASE

```

```

# -----

conn = sqlite3.connect(db_file)

customer_revenue.to_sql(
    "customer_revenue_report",
    conn,
    if_exists="replace",
    index=False
)

# -----
# 4. DISPLAY REPORT
# -----

print(
    "\nFinal Customer Revenue Table:"
)

print(
    pd.read_sql(
        "SELECT * FROM customer_revenue_report",
        conn
    ).to_string(index=False)
)

conn.close()

# =====
# TASK 3
# HISTORICAL + NEWLY ARRIVING DATA
# =====

print("\n\n===== TASK 3 =====")
print("Historical and Newly Arriving Data Pipeline")

db_file = "task3_incremental.db"

# -----
# 1. CREATE DATABASE
# -----

conn = sqlite3.connect(db_file)

cursor = conn.cursor()

# -----
# CREATE TABLE FOR HISTORICAL DATA

```

```

# -----

cursor.execute("""
CREATE TABLE user_profiles (
    user_id INTEGER PRIMARY KEY,
    email TEXT,
    status TEXT
)
""")

# -----
# HISTORICAL DATA
# -----

historical_data = [
    (1, "alice@test.com", "Active"),
    (2, "bob@test.com", "Active")
]

cursor.executemany(
    """
    INSERT INTO user_profiles
    VALUES (?, ?, ?)
    """,
    historical_data
)

conn.commit()

# -----
# DISPLAY HISTORICAL DATA
# -----

print("\nInitial Historical Data in DB:")

print(
    pd.read_sql(
        "SELECT * FROM user_profiles",
        conn
    ).to_string(index=False)
)

# -----
# 2. NEWLY ARRIVING DATA
# -----

new_data_df = pd.DataFrame({
    "user_id": [2, 3],
    "email": [

```

```

        "bob_updated@test.com",
        "charlie@test.com"
    ],
    "status": [
        "Inactive",
        "Active"
    ]
})

print("\nNewly Arriving Data:")

print(new_data_df)

# -----
# 3. INCREMENTAL LOAD
# -----

# Step A:
# Load new data into staging table

new_data_df.to_sql(
    "staging_profiles",
    conn,
    if_exists="replace",
    index=False
)

# Step B:
# UPSERT data into main table

cursor.execute("""
INSERT OR REPLACE INTO user_profiles
(
    user_id,
    email,
    status
)
SELECT
    user_id,
    email,
    status
FROM staging_profiles
""")

conn.commit()

# Step C:
# Remove staging table

```

```

cursor.execute(
    "DROP TABLE staging_profiles"
)

conn.commit()

# -----
# 4. DISPLAY FINAL DATABASE
# -----

print(
    "\nDatabase State After Incremental Load:"
)

print(
    pd.read_sql(
        "SELECT * FROM user_profiles",
        conn
    ).to_string(index=False)
)

# -----
# CLOSE DATABASE
# -----

conn.close()

# =====
# CLEANUP
# =====

if os.path.exists(csv_file):
    os.remove(csv_file)

if os.path.exists("task1_pipeline.db"):
    os.remove("task1_pipeline.db")

if os.path.exists("task2_ecommerce.db"):
    os.remove("task2_ecommerce.db")

if os.path.exists("task3_incremental.db"):
    os.remove("task3_incremental.db")

# =====
# FINAL
# =====

print("\n" + "=" * 65)
print("                ALL PRACTICAL 9 TASKS COMPLETED")
print("=" * 65)

```

=====

PRACTICAL 9 - ETL PIPELINE

=====

===== TASK 1 =====

End-to-End CSV to SQL Pipeline

Original Data:

	transaction_id	date	amount	category
0	1	2026-10-01	150.50	Electronics
1	2	NaN	200.00	Clothing
2	3	2026-10-03	ERROR	Electronics
3	4	2026-10-04	99.99	Clothing

Cleaned Data:

	transaction_id	date	amount	category
0	1	2026-10-01	150.50	Electronics
3	4	2026-10-04	99.99	Clothing

Final Aggregated Report:

category	total_sales	total_revenue
Clothing	1	99.99
Electronics	1	150.50

===== TASK 2 =====

E-Commerce ETL Pipeline

Customers:

	cust_id	name
0	1	Alice
1	2	Bob

Products:

	prod_id	price
0	101	1200.0
1	102	50.0

Orders:

	order_id	cust_id	prod_id	qty
0	1001	1	101	1
1	1002	1	102	2
2	1003	2	102	1

Payments:

	order_id	status
0	1001	Completed
1	1002	Failed
2	1003	Completed

Final Customer Revenue Table:

name	lifetime_revenue
Alice	1200.0
Bob	50.0

===== TASK 3 =====

Historical and Newly Arriving Data Pipeline

Initial Historical Data in DB:

user_id	email	status
1	alice@test.com	Active
2	bob@test.com	Active

Newly Arriving Data:

	user_id	email	status
0	2	bob_updated@test.com	Inactive
1	3	charlie@test.com	Active

Database State After Incremental Load:

user_id	email	status
1	alice@test.com	Active
2	bob_updated@test.com	Inactive
3	charlie@test.com	Active

=====

ALL PRACTICAL 9 TASKS COMPLETED

=====

Name:- Tanmay Santosh Sanap

Roll No:-40

Student Id:- 5144978

College:- B K BIRLA COLLEGE

Subject:-Data Engineering (Practical No :- 10)

End-to-End Data EngineeringMini Project:-

This comprehensive, standalone mini-project fulfills the requirements of Practical 10. While originally designed for a group of two, this scope has been expanded and adapted for a single developer to showcase a complete Data Engineering lifecycle.

The project simulates a retail company's pipeline. It includes:

- 1.Data Ingestion: Reading from diverse sources (CSV and JSON).
- 2.Data Cleaning & Transformation: Handling missing values, standardizing dates, and calculating derived metrics.
- 3.Data Warehouse Design: Implementing a Star Schema (Fact and Dimension tables).
- 4.Storage: Loading the transformed data into a relational SQLite database.
- 5.Reporting: Querying the warehouse to generate a business report and a data visualization.

```
In [3]: # =====  
# PRACTICAL 10  
# END-TO-END DATA ENGINEERING MINI PROJECT  
# =====  
  
import pandas as pd  
import sqlite3  
import json  
import os  
import matplotlib.pyplot as plt  
  
print("=" * 70)  
print("      STARTING END-TO-END DATA ENGINEERING PIPELINE")
```

```

print("=" * 70)

# =====
# PHASE 0: SETUP & RAW DATA GENERATION
# =====

print("\n[Phase 0] Generating simulated raw data...")

# -----
# 1. CUSTOMERS DATA - CSV
# -----

customers_data = """customer_id,first_name,last_name,region
1,Alice,Smith,North
2,Bob,Jones,South
3,Charlie,Brown,East
4,Diana,Prince,West
5,Eve,,North"""

with open("raw_customers.csv", "w") as f:
    f.write(customers_data)

# -----
# 2. PRODUCTS DATA - JSON
# -----

products_data = [
    {
        "product_id": 101,
        "product_name": "Laptop",
        "category": "Electronics",
        "price": 1200.00
    },
    {
        "product_id": 102,
        "product_name": "Desk Chair",
        "category": "Furniture",
        "price": 150.50
    },
    {
        "product_id": 103,
        "product_name": "Mouse",
        "category": "Electronics",
        "price": 25.00
    },
    {
        "product_id": 104,
        "product_name": "Notebook",
        "category": "Stationery",
        "price": 5.00
    }
]

```

```

]

with open("raw_products.json", "w") as f:
    json.dump(products_data, f)

# -----
# 3. TRANSACTIONS DATA - CSV
# -----

transactions_data = """tx_id,date,customer_id,product_id,quantity
1001,2026-10-01,1,101,1
1002,2026-10-02,2,102,2
1003,,3,103,1
1004,2026-10-02,1,103,2
1005,2026-10-03,4,104,-5
1006,2026-10-04,5,101,1"""

with open("raw_transactions.csv", "w") as f:
    f.write(transactions_data)

# =====
# PHASE 1: DATA INGESTION
# =====

print("\n[Phase 1] Extracting data from multiple sources...")

# Read customer CSV
df_customers = pd.read_csv(
    "raw_customers.csv"
)

# Read products JSON
df_products = pd.read_json(
    "raw_products.json"
)

# Read transactions CSV
df_transactions = pd.read_csv(
    "raw_transactions.csv"
)

print("\nCustomers Data:")
print(df_customers)

print("\nProducts Data:")
print(df_products)

print("\nTransactions Data:")

```

```

print(df_transactions)

# =====
# PHASE 2: DATA CLEANING & TRANSFORMATION
# =====

print("\n[Phase 2] Cleaning and Transforming data...")

# -----
# CLEAN CUSTOMERS
# -----

# Fill missing last names
df_customers["last_name"] = (
    df_customers["last_name"]
    .fillna("Unknown")
)

# Create full name
df_customers["full_name"] = (
    df_customers["first_name"]
    + " "
    + df_customers["last_name"]
)

# Select required columns
df_customers = df_customers[
    [
        "customer_id",
        "full_name",
        "region"
    ]
]

# -----
# CLEAN TRANSACTIONS
# -----

initial_tx_count = len(df_transactions)

# Remove transactions with missing dates
df_transactions = df_transactions.dropna(
    subset=["date"]
)

# Remove negative or zero quantities
df_transactions = df_transactions[

```

```

    df_transactions["quantity"] > 0
]

# Display number of removed records
print(
    f"> Dropped "
    f"{initial_tx_count - len(df_transactions)} "
    f"invalid transaction records."
)

# -----
# DATE TRANSFORMATION
# -----

df_transactions["date"] = pd.to_datetime(
    df_transactions["date"]
)

# =====
# CREATE DATE DIMENSION
# =====

df_date_dim = pd.DataFrame({
    "date_id":
        df_transactions["date"]
        .dt.strftime("%Y%m%d")
        .astype(int),

    "full_date":
        df_transactions["date"].dt.date,

    "year":
        df_transactions["date"].dt.year,

    "month":
        df_transactions["date"].dt.month,

    "day":
        df_transactions["date"].dt.day
})

# Remove duplicate dates
df_date_dim = df_date_dim.drop_duplicates()

# -----
# CREATE date_id IN TRANSACTIONS
# -----

```

```

df_transactions["date_id"] = (
    df_transactions["date"]
    .dt.strftime("%Y%m%d")
    .astype(int)
)

# -----
# JOIN TRANSACTIONS WITH PRODUCTS
# -----

df_transactions = df_transactions.merge(
    df_products[
        [
            "product_id",
            "price"
        ]
    ],
    on="product_id",
    how="left"
)

# -----
# CALCULATE TOTAL AMOUNT
# -----

df_transactions["total_amount"] = (
    df_transactions["quantity"]
    *
    df_transactions["price"]
)

# -----
# CREATE FACT TABLE
# -----

df_fact_sales = df_transactions[
    [
        "tx_id",
        "date_id",
        "customer_id",
        "product_id",
        "quantity",
        "total_amount"
    ]
]

print("\nCleaned Customer Data:")
print(df_customers)

print("\nDate Dimension:")

```

```

print(df_date_dim)

print("\nFact Sales:")
print(df_fact_sales)

# =====
# PHASE 3: DATA WAREHOUSE DESIGN & STORAGE
# =====

print(
    "\n[Phase 3] Loading Star Schema "
    "into SQLite Data Warehouse..."
)

db_name = "enterprise_data_warehouse.db"

# Create SQLite connection
conn = sqlite3.connect(
    db_name
)

# -----
# LOAD CUSTOMER DIMENSION
# -----

df_customers.to_sql(
    "dim_customer",
    conn,
    if_exists="replace",
    index=False
)

# -----
# LOAD PRODUCT DIMENSION
# -----

df_products.to_sql(
    "dim_product",
    conn,
    if_exists="replace",
    index=False
)

# -----
# LOAD DATE DIMENSION
# -----

df_date_dim.to_sql(

```

```

        "dim_date",
        conn,
        if_exists="replace",
        index=False
    )

# -----
# LOAD FACT TABLE
# -----

df_fact_sales.to_sql(
    "fact_sales",
    conn,
    if_exists="replace",
    index=False
)

print(
    f"> Successfully loaded "
    f"{len(df_fact_sales)} "
    f"records into fact_sales."
)

# =====
# PHASE 4: REPORTING & VISUALIZATION
# =====

print(
    "\n[Phase 4] Generating Business Reports "
    "from Data Warehouse..."
)

# =====
# REPORT 1: REVENUE BY CATEGORY
# =====

query_sales_by_category = """
SELECT
    p.category,
    SUM(f.quantity) AS items_sold,
    SUM(f.total_amount) AS total_revenue

FROM fact_sales f

JOIN dim_product p
    ON f.product_id = p.product_id

GROUP BY p.category

ORDER BY total_revenue DESC

```

```

"""

df_report1 = pd.read_sql(
    query_sales_by_category,
    conn
)

print(
    "\n--- REPORT 1: Revenue by Category ---"
)

print(
    df_report1.to_string(
        index=False
    )
)

# =====
# REPORT 2: TOP CUSTOMERS
# =====

query_top_customers = """
SELECT
    c.full_name,
    c.region,
    SUM(f.total_amount) AS lifetime_value

FROM fact_sales f

JOIN dim_customer c
    ON f.customer_id = c.customer_id

GROUP BY
    c.full_name,
    c.region

ORDER BY lifetime_value DESC
"""

df_report2 = pd.read_sql(
    query_top_customers,
    conn
)

print(
    "\n--- REPORT 2: Top Customers ---"
)

print(

```

```

        df_report2.to_string(
            index=False
        )
    )

# =====
# RED REVENUE CHART
# =====

print(
    "\nCreating Revenue Visualization..."
)

plt.figure(
    figsize=(8, 5)
)

# RED BARS
plt.bar(
    df_report1["category"],
    df_report1["total_revenue"],
    color="red"
)

plt.title(
    "Total Revenue by Product Category"
)

plt.xlabel(
    "Category"
)

plt.ylabel(
    "Revenue ($)"
)

plt.grid(
    axis="y",
    linestyle="-",
    alpha=0.7
)

plt.tight_layout()

# Save chart
chart_filename = "revenue_report_chart.png"

```

```

plt.savefig(
    chart_filename
)

# Display chart
plt.show()

print(
    f"\n> Red revenue chart saved as "
    f"'{chart_filename}'"
)

# =====
# EXPORT STAR SCHEMA TO CSV
# =====

print(
    "\nExporting Star Schema to CSV..."
)

df_fact_sales.to_csv(
    "fact_sales.csv",
    index=False
)

df_customers.to_csv(
    "dim_customer.csv",
    index=False
)

df_products.to_csv(
    "dim_product.csv",
    index=False
)

df_date_dim.to_csv(
    "dim_date.csv",
    index=False
)

print(
    "> Exported Star Schema to CSV "
    "for Power BI / Tableau."
)

```

```

# =====
# SHOW DATABASE TABLES
# =====

print("\nDatabase Tables:")

tables = pd.read_sql(
    """
    SELECT name
    FROM sqlite_master
    WHERE type='table'
    """,
    conn
)

print(tables)

# =====
# CLOSE DATABASE
# =====

conn.close()

# =====
# CLEANUP RAW INPUT FILES
# =====

if os.path.exists("raw_customers.csv"):
    os.remove("raw_customers.csv")

if os.path.exists("raw_products.json"):
    os.remove("raw_products.json")

if os.path.exists("raw_transactions.csv"):
    os.remove("raw_transactions.csv")

# =====
# FINAL MESSAGE
# =====

print("\n" + "=" * 70)

print(
    "          PIPELINE EXECUTION COMPLETE"
)

print("=" * 70)

```

```
=====
STARTING END-TO-END DATA ENGINEERING PIPELINE
=====
```

[Phase 0] Generating simulated raw data...

[Phase 1] Extracting data from multiple sources...

Customers Data:

	customer_id	first_name	last_name	region
0	1	Alice	Smith	North
1	2	Bob	Jones	South
2	3	Charlie	Brown	East
3	4	Diana	Prince	West
4	5	Eve	NaN	North

Products Data:

	product_id	product_name	category	price
0	101	Laptop	Electronics	1200.0
1	102	Desk Chair	Furniture	150.5
2	103	Mouse	Electronics	25.0
3	104	Notebook	Stationery	5.0

Transactions Data:

	tx_id	date	customer_id	product_id	quantity
0	1001	2026-10-01	1	101	1
1	1002	2026-10-02	2	102	2
2	1003	NaN	3	103	1
3	1004	2026-10-02	1	103	2
4	1005	2026-10-03	4	104	-5
5	1006	2026-10-04	5	101	1

[Phase 2] Cleaning and Transforming data...

> Dropped 2 invalid transaction records.

Cleaned Customer Data:

	customer_id	full_name	region
0	1	Alice Smith	North
1	2	Bob Jones	South
2	3	Charlie Brown	East
3	4	Diana Prince	West
4	5	Eve Unknown	North

Date Dimension:

	date_id	full_date	year	month	day
0	20261001	2026-10-01	2026	10	1
1	20261002	2026-10-02	2026	10	2
5	20261004	2026-10-04	2026	10	4

Fact Sales:

	tx_id	date_id	customer_id	product_id	quantity	total_amount
0	1001	20261001	1	101	1	1200.0
1	1002	20261002	2	102	2	301.0
2	1004	20261002	1	103	2	50.0
3	1006	20261004	5	101	1	1200.0

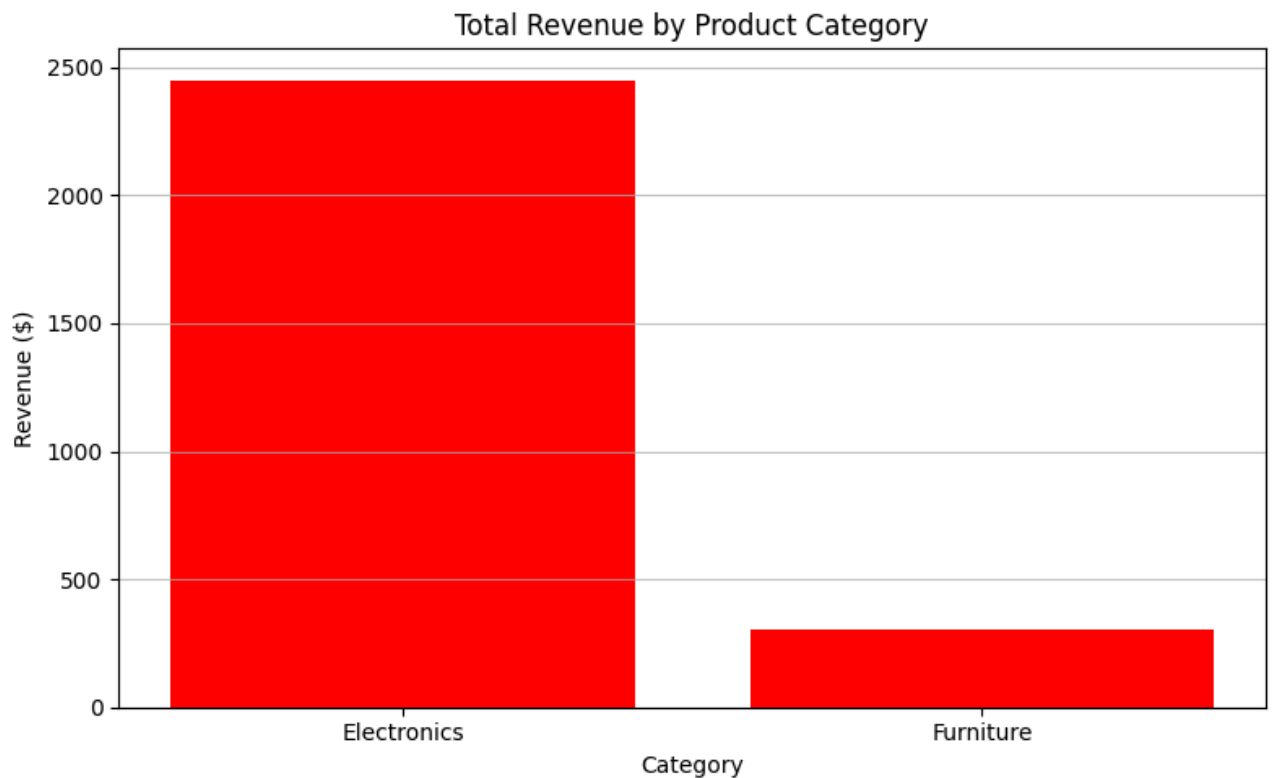
[Phase 3] Loading Star Schema into SQLite Data Warehouse...
> Successfully loaded 4 records into fact_sales.

[Phase 4] Generating Business Reports from Data Warehouse...

--- REPORT 1: Revenue by Category ---
category items_sold total_revenue
Electronics 4 2450.0
Furniture 2 301.0

--- REPORT 2: Top Customers ---
full_name region lifetime_value
Alice Smith North 1250.0
Eve Unknown North 1200.0
Bob Jones South 301.0

Creating Revenue Visualization...



> Red revenue chart saved as 'revenue_report_chart.png'

Exporting Star Schema to CSV...

> Exported Star Schema to CSV for Power BI / Tableau.

Database Tables:

	name
0	dim_customer
1	dim_product
2	dim_date
3	fact_sales

```
=====
PIPELINE EXECUTION COMPLETE
=====
```